

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO

CENTRO TECNOLÓGICO

COLEGIADO DO CURSO DE CIÊNCIA DA COMPUTAÇÃO

Áurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

Vitória, ES

2026

Áurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

Monografia apresentada ao Curso de Ciência da Computação do Centro Tecnológico da Universidade Federal do Espírito Santo, como requisito parcial para obtenção do Grau de Bacharel em Ciência da Computação.

Universidade Federal do Espírito Santo – UFES

Centro Tecnológico

Colegiado do Curso de Ciência da Computação

Orientador: Prof. Dr. João Paulo Almeida

Vitória, ES

2026

Áurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor/ Áurea Santos de Oliveira. – Vitória, ES, 2026-

83 p. : il. (algumas color.) ; 30 cm.

Orientador: Prof. Dr. João Paulo Almeida

Monografia (PG) – Universidade Federal do Espírito Santo – UFES
Centro Tecnológico
Colegiado do Curso de Ciência da Computação, 2026.

1. GraphRAG. 2. Geração aumentada por recuperação. 3. Grafos de conhecimento. 4. Código de Defesa do Consumidor. I. Almeida, João Paulo. II. Universidade Federal do Espírito Santo. IV. GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

CDU 02:141:005.7

Áurea Santos de Oliveira

GraphRAG Aplicado à Legislação Consumerista: uma Prova de Conceito para Indexação e Consulta em Grafo do Código de Defesa do Consumidor

Monografia apresentada ao Curso de Ciência da Computação do Centro Tecnológico da Universidade Federal do Espírito Santo, como requisito parcial para obtenção do Grau de Bacharel em Ciência da Computação.

Trabalho aprovado. Vitória, ES, (dia) de (mês) de (ano):

Prof. Dr. João Paulo Almeida
Orientador

Nome do Membro da Banca
Nome da Instituição

Nome do Membro da Banca
Nome da Instituição

Vitória, ES
2026

Resumo

Aluno: Versão preliminar do resumo. Completar com os resultados da avaliação (Capítulo 6) e ajustar para no máximo 500 palavras.

Sistemas de geração aumentada por recuperação (*Retrieval-Augmented Generation* – RAG) permitem que modelos de linguagem de grande porte respondam perguntas com base em uma base de conhecimento externa. Na abordagem vetorial tradicional, porém, o texto é recuperado em fragmentos isolados, o que limita respostas que dependem de relações entre partes distantes do corpus – situação típica de textos legislativos, nos quais artigos remetem uns aos outros e estabelecem definições, exceções e consequências. Este trabalho investiga o GraphRAG, *framework* de código aberto da Microsoft que constrói, com o auxílio de um modelo de linguagem, um grafo de conhecimento a partir do texto, agrupa suas entidades em comunidades hierárquicas e gera relatórios sobre essas comunidades, oferecendo quatro modalidades de consulta: local, global, DRIFT e básica (vetorial). Como prova de conceito, o Código de Defesa do Consumidor (Lei nº 8.078/1990) foi estruturado em um registro por artigo, com sua hierarquia e situação normativa, e indexado com o GraphRAG 3.1.0 utilizando modelos de linguagem em nuvem, uma taxonomia de catorze tipos de entidade definida para o domínio e *prompts* de extração ajustados ao corpus. O índice resultante contém 1.373 entidades, 6.885 relações e 369 comunidades. Foi desenvolvida ainda uma aplicação web, executada em contêineres Docker, que permite explorar o grafo, realizar consultas nas quatro modalidades acompanhando cada etapa da recuperação (contexto recuperado, *prompt* enviado ao modelo, resposta e consumo de *tokens*), destacar no grafo a parte utilizada como contexto e registrar todo o histórico em banco de dados para a avaliação. Durante o desenvolvimento foram identificados e contornados defeitos da biblioteca que afetam diretamente a comparação entre as modalidades, entre eles uma falha que faz a busca básica retornar respostas vazias quando usada programaticamente. [Resultados da avaliação comparativa entre as modalidades do GraphRAG e o *baseline* vetorial.]

Palavras-chave: GraphRAG. Geração aumentada por recuperação. Grafos de conhecimento. Modelos de linguagem. Código de Defesa do Consumidor.

Lista de ilustrações

Figura 1 – Funcionamento do RAG vetorial tradicional. 20
Figura 2 – *Workflows* do método padrão de indexação do GraphRAG 3.1.0. 28
Figura 3 – Funcionamento da busca DRIFT no GraphRAG 3.1.0: a cada rodada, perguntas de acompanhamento são respondidas por buscas locais e geram novas perguntas. 34
Figura 4 – Grafo de conhecimento do CDC na aplicação web, com as entidades coloridas por tipo. 55
Figura 5 – Arquitetura da aplicação. 59
Figura 6 – Tabelas do banco de dados da aplicação. 64
Figura 7 – Aba Grafo com as entidades coloridas por comunidade. 65
Figura 8 – Aba Consultas: formulário com modalidade, nível de comunidade, formato da resposta e perguntas de exemplo. 66
Figura 9 – Linha do tempo de uma consulta real na busca local: etapas recolhíveis, resposta com citações e métricas. 66
Figura 10 – Contexto recuperado por uma consulta real na busca básica: os trechos (artigos) recuperados por similaridade, em ordem. 67
Figura 11 – Aba Histórico com consultas reais nas buscas DRIFT, básica e local. . 67
Figura 12 – Destaque, no grafo completo, das entidades e relações usadas como contexto em uma consulta real na busca local sobre o art. 49 do CDC. 68

Lista de tabelas

Tabela 1 – Tabelas produzidas pela indexação do GraphRAG 3.1.0. 32
Tabela 2 – Comparação entre as modalidades de consulta do GraphRAG 3.1.0 (valores padrão). 35
Tabela 3 – *Prompts* do GraphRAG 3.1.0. 36
Tabela 4 – Principais opções de configuração da indexação. 37
Tabela 5 – Opções de configuração das modalidades de consulta e seus efeitos. . . 39
Tabela 6 – Defeitos e comportamentos não documentados identificados no GraphRAG 3.1.0. 41
Tabela 7 – Problemas corrigidos no *parser* da legislação. 45
Tabela 8 – Modelos utilizados em cada etapa. 47
Tabela 9 – Taxonomia de tipos de entidade utilizada. 49
Tabela 10 – Erros de configuração encontrados e corrigidos antes da indexação. . . 52
Tabela 11 – Resumo do índice gerado (versão 1). 53
Tabela 12 – Distribuição das entidades por tipo. 53
Tabela 13 – Comunidades do nível 0, com os títulos gerados pelo modelo. 54
Tabela 14 – Consumo de *tokens* da indexação. 55
Tabela 15 – Serviços do Docker Compose. 59
Tabela 16 – Módulos do *backend*. 60
Tabela 17 – Eventos emitidos durante uma consulta. 61
Tabela 18 – Rotas da API do *backend*. 63
Tabela 19 – Consultas reais de validação. 70

Lista de abreviaturas e siglas

API	*Application Programming Interface*
CDC	Código de Defesa do Consumidor (Lei nº 8.078/1990)
CLI	*Command Line Interface*
CORS	*Cross-Origin Resource Sharing*
DRIFT	*Dynamic Reasoning and Inference with Flexible Traversal*
HyDE	*Hypothetical Document Embeddings*
HTTP	*Hypertext Transfer Protocol*
JSON	*JavaScript Object Notation*
LLM	*Large Language Model*
ORM	*Object-Relational Mapping*
PLN	Processamento de Linguagem Natural
PoC	*Proof of Concept* (prova de conceito)
RAG	*Retrieval-Augmented Generation*
REST	*Representational State Transfer*
SPA	*Single Page Application*
SQL	*Structured Query Language*
SSE	*Server-Sent Events*
YAML	*YAML Ain't Markup Language*

Sumário

1 INTRODUÇÃO 12
1.1 Motivação e Justificativa 13
1.2 Objetivos 14
1.3 Método de Desenvolvimento do Trabalho 15
1.4 Organização da Monografia 16

2 REFERENCIAL TEÓRICO E TECNOLOGIAS UTILIZADAS 17
2.1 Modelos de Linguagem de Grande Porte 17
2.1.1 *Tokens* e janela de contexto 17
2.1.2 Custo 18
2.1.3 Temperatura e modelos de raciocínio 18
2.1.4 Alucinação 18
2.2 *Embeddings* e Busca Vetorial 19
2.3 Geração Aumentada por Recuperação (RAG) 19
2.4 Grafos de Conhecimento 21
2.5 Detecção de Comunidades 21
2.6 RAG Baseado em Grafos 22
2.7 O Domínio Jurídico e o Código de Defesa do Consumidor 22
2.7.1 Estrutura dos textos legislativos 22
2.7.2 O Código de Defesa do Consumidor 23
2.7.3 Processamento de linguagem natural no domínio jurídico 24
2.8 Tecnologias Utilizadas 24
2.8.1 Biblioteca GraphRAG e componentes de dados 24
2.8.2 *Backend* 25
2.8.3 *Frontend* 25
2.8.4 Infraestrutura 26

3 O *FRAMEWORK* GRAPHRAG 27
3.1 Visão Geral e Organização 27
3.2 *Pipeline* de Indexação 28
3.2.1 Leitura e segmentação do texto 29
3.2.2 Extração do grafo 29
3.2.3 Resumo de descrições 30
3.2.4 Alegações (*claims*) 30
3.2.5 Detecção de comunidades 30
3.2.6 Relatórios de comunidade 30

3.2.7 *Embeddings* . . . 31
3.2.8 Cache . . . 31
3.3 Dados Produzidos . . . **31**
3.4 Modalidades de Consulta . . . **31**
3.4.1 Busca local . . . 32
3.4.2 Busca global . . . 33
3.4.3 Busca DRIFT . . . 33
3.4.4 Busca básica . . . 34
3.4.5 Comparação entre as modalidades . . . 34
3.5 *Prompts* . . . **35**
3.5.1 Ajuste automático de *prompts* . . . 35
3.6 Configuração . . . **36**
3.6.1 Modelos . . . 37
3.6.2 Entrada, armazenamento e cache . . . 37
3.6.3 Indexação . . . 37
3.6.4 Consultas . . . 38
3.7 Defeitos e Comportamentos Não Documentados . . . **40**

4 INDEXAÇÃO DO CÓDIGO DE DEFESA DO CONSUMIDOR . . . **43**
4.1 Primeira Versão: Modelos Locais . . . **43**
4.2 Preparação do Corpus . . . **44**
4.2.1 Fonte e estratégia . . . 44
4.2.2 O *parser* da legislação . . . 44
4.2.3 Dispositivos vetados e revogados . . . 45
4.2.4 Configuração da entrada . . . 45
4.3 Segmentação . . . **46**
4.4 Escolha dos Modelos . . . **46**
4.5 Temperatura . . . **47**
4.6 Taxonomia de Entidades . . . **48**
4.7 Ajuste dos *Prompts* de Indexação . . . **49**
4.7.1 Limitações do ajuste automático . . . 49
4.7.2 *Script* próprio de ajuste . . . 50
4.7.3 Escolha dos exemplos . . . 51
4.8 Configuração Final . . . **52**
4.9 Execução da Indexação . . . **52**
4.10 O Grafo Resultante . . . **52**
4.10.1 Visão geral . . . 52
4.10.2 Exemplos . . . 53
4.10.3 Verificações de qualidade . . . 54
4.11 Custo da Indexação . . . **55**

4.12 Publicação e Reprodutibilidade . . . 56

5 APLICAÇÃO WEB PARA EXPLORAÇÃO E CONSULTA DO GRAFO 57
5.1 Objetivos e Funcionalidades . . . 57
5.2 Arquitetura . . . 58
5.3 Contêineres e Execução . . . 59
5.4 *Backend* . . . 60
5.4.1 Organização . . . 60
5.4.2 Carregamento do índice e montagem dos motores . . . 60
5.4.3 Rastreamento das etapas . . . 61
5.4.4 Execução em segundo plano e transmissão . . . 61
5.4.5 Correção da busca básica . . . 62
5.4.6 Versionamento do índice . . . 62
5.4.7 Rotas da API . . . 63
5.5 Banco de Dados . . . 63
5.6 *Frontend* . . . 64
5.6.1 Comunicação com o *backend* . . . 64
5.6.2 Aba Grafo . . . 64
5.6.3 Aba Consultas . . . 65
5.6.4 Aba Histórico . . . 66
5.6.5 Destaque do contexto no grafo . . . 67
5.7 Problemas de Ambiente . . . 69
5.8 Validação Funcional . . . 69

6 AVALIAÇÃO COMPARATIVA DAS MODALIDADES DE BUSCA . 71
6.1 Objetivo e Questões de Avaliação . . . 71
6.2 Planejamento . . . 71
6.3 Configuração das Buscas . . . 72
6.4 Resultados . . . 72
6.5 Discussão e Ameaças à Validade . . . 72

7 CONCLUSÃO . . . 73
7.1 Considerações Finais . . . 73
7.2 Trabalhos Futuros . . . 74

REFERÊNCIAS . . . 75

APÊNDICES 79

APÊNDICE A – CONFIGURAÇÃO DO GRAPHRAG UTILIZADA 80

1 Introdução

Modelos de linguagem de grande porte (*Large Language Models* – LLMs) (VASWANI et al., 2017; BROWN et al., 2020) tornaram-se capazes de responder perguntas em linguagem natural sobre praticamente qualquer assunto. Seu conhecimento, no entanto, é limitado ao que estava presente nos dados de treinamento e está sujeito a erros factuais apresentados com aparente segurança, fenômeno conhecido como alucinação (JI et al., 2023). Para aplicações em que a resposta precisa estar fundamentada em um conjunto específico de documentos – como uma lei –, a abordagem mais utilizada é a geração aumentada por recuperação (*Retrieval-Augmented Generation* – RAG) (LEWIS et al., 2020): no momento da consulta, os trechos mais relevantes de uma base de conhecimento externa são recuperados e inseridos no contexto do modelo, que gera a resposta a partir deles.

Na forma mais comum de RAG, o texto é dividido em fragmentos (*chunks*), cada fragmento é convertido em um vetor numérico (*embedding*) e, na consulta, recuperam-se os fragmentos cujos vetores são mais similares ao vetor da pergunta (GAO et al., 2023b). Essa estratégia funciona bem quando a resposta está concentrada em poucos trechos, mas apresenta limitações quando o conhecimento tem estrutura fortemente relacional e a resposta depende de conectar informações dispersas pelo corpus. Essa situação é recorrente em textos legislativos: artigos remetem uns aos outros, estabelecem definições usadas em outros pontos da lei, preveem exceções e consequências em dispositivos distintos e organizam-se em uma hierarquia de títulos, capítulos e seções que também carrega significado.

O *framework* GraphRAG, desenvolvido pela Microsoft Research (EDGE et al., 2024), propõe uma alternativa a essa limitação. Em vez de recuperar apenas fragmentos semanticamente similares, o GraphRAG utiliza um LLM para extrair do corpus um grafo de conhecimento – entidades e relações entre elas –, agrupa as entidades em comunidades hierárquicas por meio de um algoritmo de detecção de comunidades e gera relatórios textuais que resumem cada comunidade. Essas estruturas permitem modalidades de consulta que não existem no RAG vetorial: buscas locais, que partem de entidades específicas e percorrem suas relações; buscas globais, que sintetizam informação distribuída por todo o corpus a partir dos relatórios de comunidade; e a busca DRIFT, que combina as duas de forma iterativa.

Este trabalho aplica o GraphRAG ao Código de Defesa do Consumidor (CDC, Lei nº 8.078/1990) (BRASIL, 1990). O escopo está delimitado à construção de uma prova de conceito (*Proof of Concept* – PoC) composta por: (i) um *pipeline* de preparação do

texto do CDC e de indexação com o GraphRAG, que gera o grafo de conhecimento da lei; e (ii) uma aplicação web para visualização interativa desse grafo e para realização de consultas em linguagem natural nas quatro modalidades oferecidas pelo GraphRAG (local, global, DRIFT e básica), exibindo passo a passo o que cada modalidade recupera e envia ao modelo e registrando cada consulta para posterior avaliação. A modalidade básica, que realiza busca vetorial pura sobre o mesmo corpus, com os mesmos modelos, é utilizada como *baseline* de RAG tradicional. Ficam fora do escopo a extensão a outras leis, a incorporação de jurisprudência e a disponibilização do sistema em ambiente de produção.

1.1 Motivação e Justificativa

A motivação deste trabalho parte do interesse em investigar, na prática, as limitações dos sistemas de RAG tradicionais diante de textos com alta densidade de relações internas, como a legislação. Um artigo de lei raramente pode ser interpretado de forma isolada: ele depende de definições estabelecidas em outros artigos (no CDC, por exemplo, os conceitos de consumidor e fornecedor dos arts. 2º e 3º permeiam todo o código), está sujeito a exceções previstas em outros dispositivos e produz consequências tratadas em capítulos distintos, como sanções administrativas e infrações penais. Uma recuperação baseada apenas em similaridade entre a pergunta e fragmentos de texto tende a trazer trechos relacionados ao vocabulário da pergunta, mas não necessariamente a cadeia de dispositivos necessária para uma resposta completa e juridicamente coerente. Além disso, perguntas amplas, como "quais são os direitos básicos do consumidor?", exigiriam sintetizar informação dispersa por grande parte do corpus, o que não se resolve recuperando um pequeno número de fragmentos.

O CDC foi escolhido como domínio de aplicação por três razões. Primeiro, trata-se de uma legislação de grande relevância social e de ampla aplicação no cotidiano da população brasileira. Segundo, tem tamanho gerenciável para uma prova de conceito (130 artigos, cerca de 22 mil *tokens*), o que permite iterar sobre a configuração e a indexação em tempo e custo compatíveis com um Trabalho de Conclusão de Curso, sem perder a complexidade relacional típica de textos legislativos. Terceiro, o código passou por alterações relevantes ao longo do tempo – como a inclusão do tratamento do superendividamento pela Lei nº 14.181/2021 (BRASIL, 2021) – e contém dispositivos vetados e revogados, o que permite verificar se o sistema representa corretamente a situação normativa de cada dispositivo.

Do ponto de vista técnico, o trabalho se justifica pela distância entre a proposta do GraphRAG, apresentada em artigo científico e em documentação, e o seu uso efetivo em um domínio específico. Embora sistemas de RAG aplicados ao domínio jurídico já tenham sido explorados (CUI et al., 2023; MAGESH et al., 2024), a maior parte utiliza recuperação vetorial tradicional, e há pouca documentação prática sobre como adaptar o

GraphRAG a um corpus jurídico em português: como estruturar o texto de entrada, como definir os tipos de entidade, como ajustar os *prompts*, quanto custa a indexação, como se comportam e quanto custam as diferentes modalidades de consulta. Como se verá nos capítulos seguintes, essa adaptação revelou também comportamentos não documentados e defeitos da própria biblioteca que afetam diretamente qualquer comparação entre as modalidades – o que reforça a importância de uma investigação prática e transparente.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver uma prova de conceito de um sistema de *Retrieval-Augmented Generation* baseado em grafo de conhecimento (GraphRAG) para indexação e consulta do Código de Defesa do Consumidor, disponibilizando uma aplicação web que permita a visualização do grafo de conhecimento construído e a realização de consultas nas diferentes modalidades de busca, e avaliar comparativamente essas modalidades em relação a uma abordagem de RAG vetorial tradicional.

São objetivos específicos deste trabalho:

- Revisar a literatura sobre modelos de linguagem, geração aumentada por recuperação, RAG baseado em grafos de conhecimento e aplicações no domínio jurídico;
- Estudar o *framework* GraphRAG em profundidade – seu *pipeline* de indexação, suas modalidades de consulta e suas opções de configuração –, documentando os impactos de cada opção em qualidade e custo;
- Preparar o texto do CDC para indexação, preservando a hierarquia da lei e a situação normativa (vigente, vetado ou revogado) de cada dispositivo;
- Definir uma taxonomia de tipos de entidade adequada ao CDC e ajustar os *prompts* de extração do GraphRAG ao domínio;
- Executar o *pipeline* de indexação do GraphRAG sobre o CDC e analisar o grafo de conhecimento resultante;
- Desenvolver uma aplicação web para visualização interativa do grafo e realização de consultas nas modalidades local, global, DRIFT e básica, exibindo as etapas intermediárias de cada consulta e registrando-as em banco de dados;
- Avaliar comparativamente a qualidade, a completude e o custo das respostas obtidas pelas modalidades do GraphRAG em relação ao *baseline* vetorial.

1.3 Método de Desenvolvimento do Trabalho

O trabalho caracteriza-se como uma pesquisa aplicada, de natureza tecnológica, cujo produto é uma prova de conceito funcional. O desenvolvimento foi conduzido de forma iterativa, nas seguintes etapas:

1. **Revisão bibliográfica:** levantamento de referências sobre LLMs, RAG, RAG baseado em grafos (com foco no artigo e na documentação do GraphRAG) e aplicações de PLN no domínio jurídico.

2. **Estudo do *framework*:** leitura da documentação e do código-fonte do GraphRAG (versão 3.1.0, fixada para garantir a reprodutibilidade), com o objetivo de compreender cada etapa da indexação e das consultas e o efeito de cada opção de configuração. O estudo do código foi necessário porque parte do comportamento relevante para este trabalho não está descrita na documentação.

3. **Primeira versão local:** montagem de um ambiente em contêineres Docker com modelos de código aberto executados localmente (Ollama (OLLAMA, 2026)), indexação de teste e primeira versão do visualizador do grafo. Essa etapa mostrou que, sem GPU, a indexação levava horas e a extração do grafo tinha qualidade insuficiente, o que motivou a migração para modelos em nuvem.

4. **Preparação do corpus:** desenvolvimento de um *parser* que converte o texto oficial do CDC em um registro estruturado por artigo, com a hierarquia da lei e a situação normativa de cada dispositivo, e definição da segmentação (*chunking*) do texto.

5. **Configuração e ajuste da indexação:** escolha dos modelos de linguagem por etapa do *pipeline*, com base em custo e benefício; definição da taxonomia de tipos de entidade; geração e ajuste dos *prompts* de extração; testes prévios dos modelos e da configuração.

6. **Indexação e análise do grafo:** execução do *pipeline* completo, verificação da qualidade das entidades, relações e comunidades extraídas e registro do consumo de *tokens*. O índice resultante foi publicado como versão (*release*) do repositório, para que possa ser reutilizado sem nova indexação.

7. **Desenvolvimento da aplicação web:** implementação de um *backend* que executa as consultas com a própria biblioteca GraphRAG, transmite cada etapa ao navegador e registra tudo em banco de dados, e de um *frontend* para exploração do grafo, execução de consultas, consulta ao histórico e destaque, no grafo, do contexto utilizado em cada consulta.

8. **Avaliação:** definição de um conjunto de perguntas sobre o CDC e comparação das

respostas das modalidades do GraphRAG com o *baseline* vetorial, considerando qualidade, completude, consistência entre repetições e custo.

Todas as implementações, decisões, problemas encontrados e medições foram registrados ao longo do desenvolvimento em um documento de registro de implementação mantido no repositório do projeto, que serviu de base para a redação desta monografia. O código-fonte, a configuração, os *prompts* e a documentação estão disponíveis no repositório do projeto.[1]

1.4 Organização da Monografia

Além desta introdução, esta monografia está organizada da seguinte forma:

- O Capítulo 2 apresenta o referencial teórico – modelos de linguagem, *embeddings*, RAG, grafos de conhecimento, detecção de comunidades, RAG baseado em grafos e o domínio jurídico – e as tecnologias utilizadas;
- O Capítulo 3 descreve em detalhes o *framework* GraphRAG: sua arquitetura, o *pipeline* de indexação, os dados produzidos, as modalidades de consulta, os *prompts* e as opções de configuração, com seus impactos em qualidade e custo;
- O Capítulo 4 apresenta a aplicação do GraphRAG ao CDC: preparação do corpus, escolha dos modelos, taxonomia, ajuste dos *prompts*, execução da indexação e análise do grafo resultante;
- O Capítulo 5 apresenta a aplicação web desenvolvida: funcionalidades, arquitetura, contêineres, *backend*, banco de dados e *frontend*;
- O Capítulo 6 apresenta a avaliação comparativa entre as modalidades de busca;
- O Capítulo 7 apresenta as considerações finais, as limitações do trabalho e as perspectivas de trabalhos futuros;
- O Apêndice A reproduz a configuração completa do GraphRAG utilizada.

[1] <https://github.com/aureaooliveira/graphrag-docker>

2 Referencial Teórico e Tecnologias Utilizadas

Este capítulo apresenta os fundamentos teóricos e as tecnologias que sustentam o trabalho. A Seção 2.1 introduz os modelos de linguagem de grande porte e os conceitos de *token*, janela de contexto, temperatura e custo, que são recorrentes nos capítulos seguintes. A Seção 2.2 trata de *embeddings* e busca vetorial. A Seção 2.3 apresenta a geração aumentada por recuperação e suas limitações. As seções 2.4 e 2.5 tratam de grafos de conhecimento e de detecção de comunidades, que fundamentam o RAG baseado em grafos, apresentado na Seção 2.6. A Seção 2.7 discute o domínio jurídico e a estrutura do Código de Defesa do Consumidor. Por fim, a Seção 2.8 apresenta as tecnologias utilizadas no desenvolvimento.

2.1 Modelos de Linguagem de Grande Porte

Modelos de linguagem de grande porte (LLMs) são redes neurais treinadas para estimar a probabilidade da próxima unidade de texto dada uma sequência anterior. Os modelos atuais baseiam-se na arquitetura *Transformer* (VASWANI et al., 2017), cujo mecanismo de atenção permite que cada posição da sequência considere todas as demais, e são treinados sobre grandes volumes de texto. A partir de certa escala, esses modelos passaram a resolver tarefas variadas – responder perguntas, resumir, extrair informação, traduzir – apenas a partir de instruções e de alguns exemplos fornecidos no próprio texto de entrada, sem treinamento adicional (BROWN et al., 2020). Essa capacidade de aprender a tarefa pelos exemplos do *prompt* (*few-shot learning*) é a base de todas as etapas do GraphRAG que utilizam LLMs: a extração de entidades e relações, por exemplo, é feita fornecendo ao modelo instruções e exemplos de extração seguidos do texto a ser processado.

2.1.1 *Tokens* e janela de contexto

Os modelos não processam caracteres nem palavras, mas *tokens*: unidades de texto obtidas por algoritmos de segmentação em subpalavras, como o *Byte Pair Encoding* (SENNRICH; HADDOW; BIRCH, 2016). Palavras frequentes tornam-se um único *token*, enquanto palavras raras são divididas em vários. Em português, um *token* corresponde, em média, a três ou quatro caracteres. Neste trabalho, utiliza-se a codificação `o200k_base`, a mesma dos modelos da OpenAI empregados, para medir o tamanho dos textos.

O número máximo de *tokens* que o modelo pode considerar em uma chamada – somando instruções, contexto, pergunta e resposta – é chamado de janela de contexto. Mesmo com janelas grandes, o desempenho dos modelos não é uniforme ao longo do

contexto: informações posicionadas no meio de contextos longos tendem a ser menos aproveitadas do que as posicionadas no início ou no fim (LIU et al., 2024). Por isso, sistemas de RAG não se limitam a preencher toda a janela disponível, mas definem orçamentos de contexto e selecionam o que inserir.

2.1.2 Custo

Modelos oferecidos como serviço são cobrados por *token*, com preços distintos para os *tokens* de entrada (o *prompt*) e os de saída (o texto gerado); os de saída costumam ser várias vezes mais caros. O custo de uma operação é, portanto, aproximadamente o número de chamadas ao modelo multiplicado pelo tamanho de cada chamada. Essa relação é central neste trabalho: as modalidades de consulta do GraphRAG diferem justamente no número de chamadas que fazem e no tamanho do contexto de cada uma, e o custo da indexação é dominado pelas chamadas de extração do grafo.

2.1.3 Temperatura e modelos de raciocínio

Na geração, o modelo produz, a cada passo, uma distribuição de probabilidade sobre os *tokens* possíveis, da qual o próximo *token* é amostrado. A temperatura é um parâmetro que reescala essa distribuição: valores próximos de zero tornam a escolha quase determinística (sempre o *token* mais provável), e valores maiores aumentam a diversidade do texto gerado (HOLTZMAN et al., 2020). Temperaturas baixas são tradicionalmente usadas em tarefas de extração e em avaliações, por favorecerem a reprodutibilidade.

Uma classe mais recente de modelos, chamados modelos de raciocínio, é treinada para produzir uma cadeia de raciocínio interna antes da resposta final – ideia relacionada ao *chain-of-thought prompting* (WEI et al., 2022). Nesses modelos, os *tokens* de raciocínio também são cobrados como *tokens* de saída, mas não aparecem no texto devolvido, e parâmetros de amostragem como a temperatura frequentemente não podem ser alterados. Os modelos utilizados neste trabalho pertencem a essa classe e aceitam apenas temperatura igual a 1, restrição que teve consequências metodológicas discutidas no Capítulo 4.

2.1.4 Alucinação

Alucinação é a geração de conteúdo fluente e plausível, porém não fundamentado na entrada ou nos fatos (JI et al., 2023). No domínio jurídico, o problema é particularmente grave: citar um artigo inexistente ou atribuir a um dispositivo um conteúdo que ele não possui compromete a utilidade da resposta. Mesmo ferramentas comerciais de pesquisa jurídica baseadas em RAG apresentam taxas relevantes de respostas incorretas ou sem fundamentação (MAGESH et al., 2024). Fundamentar a resposta em trechos recuperados e exigir que o modelo cite suas fontes – como fazem os *prompts* do GraphRAG – reduz,

mas não elimina, esse risco.

2.2 *Embeddings* e Busca Vetorial

Um *embedding* é uma representação de um texto como um vetor de números reais de dimensão fixa, produzida por um modelo treinado de forma que textos de significado próximo resultem em vetores próximos (REIMERS; GUREVYCH, 2019). A proximidade é geralmente medida pela similaridade de cosseno:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \tag{2.1}$$

que vale 1 para vetores de mesma direção e diminui à medida que os vetores se afastam. Com *embeddings*, a busca por significado reduz-se a encontrar os vetores mais próximos do vetor da consulta. Modelos de recuperação densa treinados para esse fim superam métodos baseados em palavras-chave em diversas tarefas de perguntas e respostas (KARPUKHIN et al., 2020).

Para coleções grandes, a busca exata pelos vizinhos mais próximos é custosa, e são utilizados índices de busca aproximada e bancos de dados vetoriais especializados (JOHNSON; DOUZE; JÉGOU, 2019). O GraphRAG utiliza por padrão o LanceDB (LANCEDB, 2024), um banco vetorial embarcado que armazena os vetores em arquivos locais, sem necessidade de um servidor. Neste trabalho, os *embeddings* são gerados pelo modelo `text-embedding-3-large` da OpenAI (OPENAI, 2024), que produz vetores de 3.072 dimensões.

2.3 Geração Aumentada por Recuperação (RAG)

O paradigma de *Retrieval-Augmented Generation* (LEWIS et al., 2020) combina um mecanismo de recuperação de informação a um modelo gerador. Em vez de depender exclusivamente do conhecimento armazenado nos pesos do modelo, o sistema recupera, no momento da consulta, os trechos mais relevantes de uma base de conhecimento externa e os inclui no *prompt*. Com isso, o conhecimento do sistema pode ser atualizado sem novo treinamento, as respostas podem ser restritas a uma fonte confiável e cada afirmação pode ser rastreada até o trecho que a fundamenta (GAO et al., 2023b).

Na forma mais simples, chamada de RAG vetorial ou ingênuo (*naive RAG*), o processo tem duas fases (Figura 1). Na **indexação**, os documentos são divididos em fragmentos (*chunks*) de tamanho limitado, geralmente com uma pequena sobreposição entre fragmentos consecutivos, e cada fragmento é convertido em um *embedding* e armazenado em um banco vetorial. Na **consulta**, a pergunta é convertida em *embedding*, os k fragmentos

mais similares são recuperados e inseridos em um *prompt* junto com instruções e a pergunta, e o modelo gera a resposta.

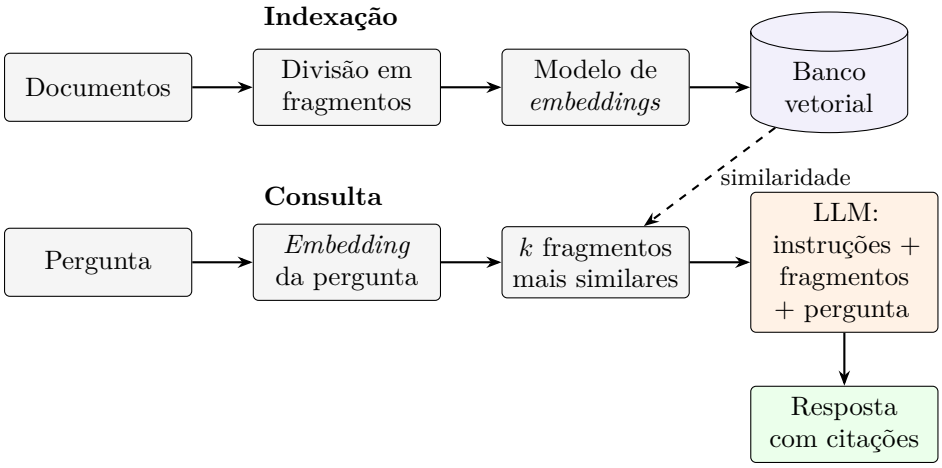


Figura 1 – Funcionamento do RAG vetorial tradicional.

Diversas variações procuram melhorar a recuperação. Uma delas, relevante para este trabalho por ser usada na busca DRIFT do GraphRAG, é a técnica HyDE (*Hypothetical Document Embeddings*) (GAO et al., 2023a): em vez de buscar com o *embedding* da pergunta, o modelo primeiro escreve uma resposta hipotética, e o *embedding* dessa resposta é usado na busca, por estar mais próximo, em forma e vocabulário, dos documentos procurados.

Apesar de eficaz em muitos cenários, o RAG vetorial apresenta limitações relevantes para este trabalho:

- **Fragmentos isolados.** A recuperação considera cada fragmento separadamente, sem representar as relações entre partes do corpus. Perguntas que exigem conectar vários trechos (raciocínio em múltiplos saltos, ou *multi-hop*) dependem de que todos esses trechos sejam, individualmente, similares à pergunta.

- **Perguntas globais.** Perguntas sobre o corpus como um todo – "quais são os principais temas?", "quais são os direitos do consumidor?" – exigiriam sintetizar informação dispersa por muitos fragmentos, o que não cabe em um pequeno conjunto recuperado. Edge et al. (2024) caracterizam essas perguntas como tarefas de sumarização focada na consulta (*query-focused summarization*), para as quais o RAG vetorial não é adequado.

- **Similaridade não é relevância.** Fragmentos que compartilham o vocabulário da pergunta podem ser recuperados à frente do trecho que efetivamente responde a ela. No domínio jurídico, em que muitos artigos usam os mesmos termos ("consumidor", "fornecedor", "produto"), esse efeito é acentuado.

2.4 Grafos de Conhecimento

Um grafo de conhecimento representa conhecimento como um grafo cujos nós são entidades do domínio e cujas arestas são relações entre elas (HOGAN et al., 2021). Na variante de grafo de propriedades, nós e arestas podem carregar atributos – tipo, descrição, peso. Grafos de conhecimento permitem consultas que percorrem relações ("quais obrigações decorrem deste artigo, e quais sanções se aplicam ao seu descumprimento?") e oferecem uma representação explícita e inspecionável do conteúdo do corpus.

Tradicionalmente, a construção de grafos de conhecimento a partir de texto exigia sistemas de extração de informação treinados para cada domínio. LLMs tornaram possível extrair entidades e relações a partir de instruções e de poucos exemplos, o que aproximou as duas áreas: grafos podem complementar LLMs com conhecimento estruturado e verificável, e LLMs podem construir e consultar grafos (PAN et al., 2024). O custo dessa abordagem é que a qualidade do grafo depende da qualidade da extração feita pelo modelo – que pode omitir entidades, criar entidades duplicadas com nomes diferentes ou relações sem fundamento no texto – e que cada fragmento do corpus exige ao menos uma chamada ao modelo.

2.5 Detecção de Comunidades

Em grafos reais, é comum que os nós se organizem em grupos densamente conectados internamente e esparsamente conectados entre si, chamados comunidades (FORTUNATO, 2010). A detecção de comunidades busca particionar o grafo nesses grupos. Uma medida clássica da qualidade de uma partição é a modularidade (NEWMAN, 2006), que compara a fração de arestas internas às comunidades com a fração esperada em um grafo aleatório com os mesmos graus:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{2.2}$$

em que A é a matriz de adjacência, k_i é o grau do nó i, m é o número de arestas e $\delta(c_i, c_j)$ vale 1 se os nós i e j estão na mesma comunidade.

O algoritmo de Louvain (BLONDEL et al., 2008) otimiza a modularidade de forma gulosa e hierárquica: move nós entre comunidades enquanto houver ganho e, em seguida, agrega cada comunidade em um único nó e repete o processo. O algoritmo de Leiden (TRAAG; WALTMAN; ECK, 2019) corrige um defeito do Louvain, que pode produzir comunidades mal conectadas ou até desconexas, com uma etapa de refinamento que garante comunidades bem conectadas. O GraphRAG utiliza uma versão hierárquica do Leiden: comunidades maiores que um tamanho máximo são novamente particionadas, formando

uma hierarquia em que o nível 0 contém poucas comunidades amplas e os níveis seguintes, comunidades progressivamente menores e mais específicas.

2.6 RAG Baseado em Grafos

O RAG baseado em grafos (*Graph RAG*) reúne abordagens que utilizam grafos na recuperação: o conhecimento é organizado como grafo, e a recuperação seleciona nós, arestas, caminhos ou subgrafos relevantes para a pergunta, que são convertidos em texto e inseridos no *prompt* (PENG et al., 2024). Em relação ao RAG vetorial, essas abordagens preservam explicitamente a estrutura relacional do corpus, permitem percorrer relações em múltiplos saltos e possibilitam resumos sobre grupos de entidades.

O GraphRAG da Microsoft (EDGE et al., 2024) é uma dessas abordagens, voltada especificamente para corpora de texto não estruturado. Seu processo pode ser resumido em quatro passos: (i) um LLM extrai entidades e relações de cada fragmento do corpus, formando um grafo de conhecimento; (ii) o grafo é particionado em uma hierarquia de comunidades com o algoritmo de Leiden; (iii) o LLM gera, para cada comunidade, um relatório que resume suas entidades, relações e temas; e (iv) as consultas utilizam essas estruturas. Para perguntas globais, cada relatório de comunidade gera uma resposta parcial, e as respostas parciais são combinadas em uma resposta final (*map-reduce*). Os autores mostraram que essa abordagem produz respostas mais abrangentes e diversas que o RAG vetorial para perguntas globais sobre corpora da ordem de um milhão de *tokens*, com avaliação feita por um LLM em comparações par a par. A biblioteca evoluiu depois do artigo, incorporando a busca local (centrada em entidades) e a busca DRIFT (RESEARCH, 2024), que combina as buscas global e local. O *framework* é descrito em detalhes no Capítulo 3.

2.7 O Domínio Jurídico e o Código de Defesa do Consumidor

2.7.1 Estrutura dos textos legislativos

A elaboração e a redação das leis brasileiras seguem a Lei Complementar nº 95/1998 (BRASIL, 1998), que define a estrutura dos textos legislativos. A unidade básica é o artigo, que pode se desdobrar em parágrafos, incisos, alíneas e itens. Os artigos podem ser agrupados em seções e subseções; estas, em capítulos; capítulos, em títulos; títulos, em livros; e livros, em partes. A mesma lei define regras de numeração: os artigos são numerados com ordinal até o nono ("Art. 9º") e com cardinal a partir do décimo ("Art. 10."). Artigos acrescentados por leis posteriores entre dois artigos existentes recebem o número do anterior seguido de letra maiúscula ("Art. 54-A"), sem renumeração dos demais. Dispositivos vetados pelo Presidente da República ou revogados por lei posterior

permanecem no texto compilado com as indicações "(Vetado)" ou "(Revogado)".

Essa estrutura tem duas consequências importantes para o processamento automático. A primeira é que ela é regular o suficiente para ser reconhecida por expressões regulares, permitindo extrair cada artigo com a sua localização na hierarquia. A segunda é que a hierarquia carrega significado: um artigo localizado no capítulo "Das Práticas Abusivas" trata de práticas abusivas mesmo que o termo não apareça em seu texto. Preservar essa informação ao dividir o texto em fragmentos é, portanto, relevante para a qualidade da recuperação e da extração.

2.7.2 O Código de Defesa do Consumidor

O Código de Defesa do Consumidor (Lei nº 8.078, de 11 de setembro de 1990) (BRASIL, 1990) estabelece normas de proteção e defesa do consumidor, de ordem pública e interesse social. O texto compilado utilizado neste trabalho tem 130 artigos – os 119 artigos originais e 11 artigos acrescentados posteriormente, identificados por letras (42-A, 54-A a 54-G e 104-A a 104-C) –, organizados em seis títulos:

- **Título I – Dos Direitos do Consumidor** (arts. 1º a 60): disposições gerais e conceitos de consumidor, fornecedor, produto e serviço; política nacional das relações de consumo; direitos básicos (art. 6º); qualidade de produtos e serviços, responsabilidade pelo fato e pelo vício, decadência e prescrição; práticas comerciais (oferta, publicidade, práticas abusivas, cobrança de dívidas, bancos de dados); proteção contratual (cláusulas abusivas, contratos de adesão); prevenção e tratamento do superendividamento (arts. 54-A a 54-G, incluídos pela Lei nº 14.181/2021 (BRASIL, 2021)); e sanções administrativas;
- **Título II – Das Infrações Penais** (arts. 61 a 80): crimes contra as relações de consumo;
- **Título III – Da Defesa do Consumidor em Juízo** (arts. 81 a 104-C): ações coletivas, coisa julgada e conciliação no superendividamento;
- **Título IV – Do Sistema Nacional de Defesa do Consumidor** (arts. 105 e 106);
- **Título V – Da Convenção Coletiva de Consumo** (arts. 107 e 108);
- **Título VI – Disposições Finais** (arts. 109 a 119).

O CDC é um exemplo típico de texto de alta densidade relacional: os conceitos definidos nos primeiros artigos são usados em todo o código; direitos do Título I correspondem a obrigações do fornecedor, a sanções administrativas e a crimes do Título II; e a

tutela coletiva do Título III remete a outras leis, como a Lei da Ação Civil Pública. O texto também contém doze artigos inteiramente vetados e diversos parágrafos e incisos vetados dentro de artigos vigentes.

2.7.3 Processamento de linguagem natural no domínio jurídico

O domínio jurídico tem vocabulário e estilo próprios, o que motivou modelos de linguagem especializados, como o LEGAL-BERT (CHALKIDIS et al., 2020), e sistemas que combinam LLMs a bases de conhecimento jurídicas, como o ChatLaw (CUI et al., 2023). Com os LLMs de uso geral, o RAG tornou-se a abordagem predominante para assistentes jurídicos, por permitir fundamentar as respostas em legislação e jurisprudência atualizadas. A avaliação de ferramentas comerciais desse tipo mostrou, contudo, que a recuperação de trechos relevantes não garante respostas corretas: as ferramentas avaliadas ainda produziam afirmações incorretas ou com citações que não as sustentavam (MAGESH et al., 2024). Esses resultados motivam investigar estratégias de recuperação que representem melhor a estrutura relacional do direito, como a proposta neste trabalho.

2.8 Tecnologias Utilizadas

Como o foco do trabalho é a investigação do GraphRAG, as tecnologias de apoio foram escolhidas priorizando a integração direta com a biblioteca, a simplicidade e a reprodutibilidade. Esta seção apresenta brevemente cada uma; a forma como foram utilizadas é descrita nos capítulos 4 e 5.

2.8.1 Biblioteca GraphRAG e componentes de dados

- **GraphRAG** (MICROSOFT, 2026a; MICROSOFT, 2026b): biblioteca Python de código aberto (licença MIT) que implementa a indexação e as consultas descritas no Capítulo 3. É utilizada pela linha de comando (*Command Line Interface* – CLI), para indexar e ajustar *prompts*, e como biblioteca, pelo *backend* da aplicação. Este trabalho utiliza a versão 3.1.0, fixada em todos os ambientes.

- **LiteLLM** (BERRIAI, 2026): biblioteca que oferece uma interface única para APIs de diversos provedores de modelos (OpenAI, Anthropic, Google, Ollama, entre outros). O GraphRAG a utiliza para todas as chamadas a modelos, o que permite trocar de provedor alterando apenas a configuração.

- **Apache Parquet** (Apache Software Foundation, 2026): formato de arquivo colunar em que o GraphRAG grava as tabelas produzidas pela indexação (documentos, fragmentos, entidades, relações, comunidades e relatórios).

- **LanceDB** (LANCEDB, 2024): banco vetorial embarcado em que o GraphRAG armazena os *embeddings*.
- **Modelos da OpenAI** (OPENAI, 2026): modelos de linguagem e de *embeddings* acessados por API, escolhidos por etapa conforme descrito na Seção 4.4.
- **Ollama** (OLLAMA, 2026): ferramenta para executar modelos de código aberto localmente, utilizada na primeira versão do projeto e mantida como configuração alternativa.

2.8.2 *Backend*

- **Python** e **FastAPI** (RAMÍREZ, 2026): o *backend* é escrito em Python por ser a linguagem do GraphRAG, o que permite usar a biblioteca diretamente, sem intermediários. O FastAPI é um *framework* web assíncrono, adequado a operações demoradas de entrada e saída como chamadas a modelos, e gera automaticamente a documentação da API.
- **REST** (FIELDING, 2000): estilo arquitetural adotado na API, em que recursos (consultas, histórico, grafo) são acessados por URLs e métodos HTTP.
- ***Server-Sent Events*** (SSE) (WHATWG, 2026): padrão da web para que o servidor envie uma sequência de eventos ao navegador em uma única conexão HTTP. É utilizado para transmitir, à medida que acontecem, as etapas de cada consulta.
- **SQLite** (SQLite Consortium, 2026) e **SQLAlchemy** (BAYER, 2026): o SQLite é um banco de dados relacional embarcado, armazenado em um único arquivo, sem servidor; o SQLAlchemy é a biblioteca de mapeamento objeto-relacional (ORM) utilizada para definir as tabelas e acessá-las a partir do Python.

2.8.3 *Frontend*

- **React** (PLATFORMS, 2026) e **TypeScript**: o *frontend* é uma aplicação de página única (*Single Page Application* – SPA) construída com React, biblioteca de componentes de interface, e escrita em TypeScript, que acrescenta tipagem estática ao JavaScript.
- **Vite** (Vite Team, 2026): ferramenta de desenvolvimento e empacotamento. Além de servir a aplicação, seu servidor de desenvolvimento atua como *proxy* para a API, como descrito no Capítulo 5.
- **Tailwind CSS** (Tailwind Labs, 2026) e componentes no estilo **shadcn/ui**: estilização por classes utilitárias e componentes de interface reutilizáveis.

- **react-force-graph** (ASTURIANO, 2026): componente de visualização de grafos com leiaute por simulação de forças, desenhado em *canvas*, capaz de exibir os mais de mil nós e seis mil arestas do grafo do CDC com interação (zoom, arraste, seleção).

2.8.4 Infraestrutura

- **Docker** e **Docker Compose** (Docker Inc., 2026): cada componente (ambiente de indexação, *backend*, *frontend* e, opcionalmente, o servidor de modelos locais) é executado em um contêiner, com dependências e versões definidas em arquivos de imagem. O Docker Compose descreve e orquestra os serviços em um único arquivo, o que garante que o sistema seja executado da mesma forma em qualquer máquina.
- **Git** e **GitHub**: controle de versão do código, da configuração e dos *prompts*, e publicação do índice gerado como versão (*release*) do repositório.

3 O *Framework* GraphRAG

Este capítulo descreve em detalhes o GraphRAG na versão 3.1.0, utilizada neste trabalho. Além da documentação oficial (MICROSOFT, 2026b), a descrição baseia-se na leitura do código-fonte da biblioteca (MICROSOFT, 2026a), necessária porque parte do comportamento relevante – como a divisão do contexto entre tipos de registro ou a forma como a busca DRIFT escolhe as perguntas que aprofunda – não está documentada. A Seção 3.1 apresenta a organização da biblioteca. As seções 3.2 e 3.3 descrevem o *pipeline* de indexação e os dados que ele produz. A Seção 3.4 detalha as quatro modalidades de consulta. A Seção 3.5 trata dos *prompts* e do ajuste automático de *prompts*. A Seção 3.6 apresenta as opções de configuração e seus efeitos em qualidade e custo, e a Seção 3.7 reúne os defeitos e comportamentos não documentados identificados durante o trabalho.

3.1 Visão Geral e Organização

O GraphRAG é distribuído como pacote Python sob licença MIT. A partir da versão 3, a biblioteca foi dividida em pacotes menores, instalados em conjunto: `graphrag` (*pipeline* de indexação, motores de consulta, ajuste de *prompts*, API e CLI), `graphrag_llm` (acesso a modelos, por meio do LiteLLM (BERRIAI, 2026)), `graphrag_chunking` (segmentação de texto), `graphrag_input` (leitura dos documentos de entrada), `graphrag_storage` e `graphrag_cache` (armazenamento de arquivos e cache), `graphrag_vectors` (bancos vetoriais) e `graphrag_common` (configuração e utilitários).

O uso mais comum é pela linha de comando, que oferece os comandos:

- `graphrag init`: cria a estrutura de um projeto, com um `settings.yaml`, um arquivo `.env` para as chaves de API e os *prompts* padrão;
- `graphrag index`: executa o *pipeline* de indexação;
- `graphrag update`: atualiza um índice existente com novos documentos, sem reprocessar os anteriores;
- `graphrag prompt-tune`: gera *prompts* de indexação adaptados ao corpus (Seção 3.5.1);
- `graphrag query`: executa uma consulta em uma das modalidades (`-method local`, `global`, `drift` ou `basic`).

Um projeto do GraphRAG é uma pasta raiz (passada com a opção `-root`) com a estrutura padrão mostrada na Listagem 3.1. Toda a configuração fica no `settings.yaml`;

os *prompts* ficam em arquivos de texto referenciados pela configuração; e todo o resultado da indexação fica na pasta `output`. As mesmas funções da CLI estão disponíveis como API Python (`graphrag.api`), e os motores de consulta podem ser montados diretamente a partir de suas classes, como faz o *backend* deste trabalho (Capítulo 5).

Listagem 3.1 – Estrutura da pasta raiz de um projeto GraphRAG (a pasta `data/` deste trabalho).

```
data/
  settings.yaml        configuração completa (modelos, indexação e consultas)
  .env                 chaves de API (fora do controle de versão)
  prompts/             prompts de indexação e de consulta (arquivos .txt)
  input/               documentos de entrada (texto, CSV ou JSON)
  output/              resultado da indexação: tabelas .parquet e banco vetorial
      LanceDB
  cache/               respostas dos modelos, reaproveitadas em novas execuções
  logs/                registros de execução
```

3.2 *Pipeline* de Indexação

A indexação é organizada como uma sequência de *workflows*, cada um responsável por uma etapa, que leem e gravam tabelas no armazenamento de saída. A biblioteca oferece três sequências pré-definidas: o método **padrão** (*standard*), em que o grafo é extraído por um LLM; o método **rápido** (*fast*), em que entidades são substituídas por sintagmas nominais extraídos com ferramentas tradicionais de PLN e as relações, por coocorrência; e o método de **atualização** (*update*), que incorpora novos documentos a um índice existente. A sequência de *workflows* pode ainda ser definida manualmente na configuração. Este trabalho utiliza o método padrão, cujas etapas são mostradas na Figura 2 e descritas a seguir.

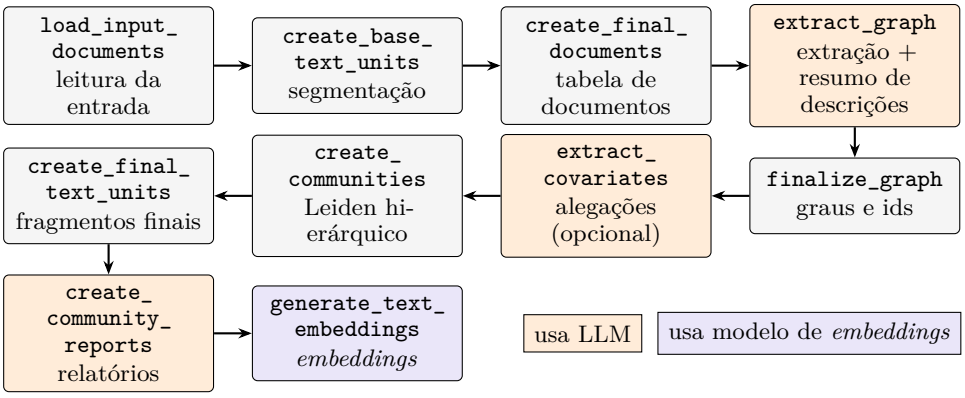


Figura 2 – *Workflows* do método padrão de indexação do GraphRAG 3.1.0.

3.2.1 Leitura e segmentação do texto

Os documentos de entrada podem ser arquivos de texto, CSV ou JSON. Nos formatos estruturados, a configuração indica qual campo contém o identificador, o título e o texto de cada documento; os demais campos ficam disponíveis como metadados. Cada documento é então dividido em unidades de texto (*text units*) de tamanho fixo em *tokens* (padrão: 1.200), com sobreposição entre unidades consecutivas (padrão: 100). Opcionalmente, campos de metadados do documento podem ser inseridos no início de cada unidade (`prepend_metadata`), de modo que o modelo veja, junto com o trecho, informações como a sua localização no documento. As unidades de texto são a granularidade de todo o processamento seguinte: cada uma é enviada separadamente ao modelo na extração, e é a elas que as entidades e relações extraídas ficam vinculadas.

3.2.2 Extração do grafo

Na etapa `extract_graph`, cada unidade de texto é inserida no *prompt* de extração, que contém instruções, a lista de tipos de entidade permitidos, exemplos de extração e o texto a processar. O modelo responde em um formato textual delimitado: cada entidade é emitida com nome, tipo e descrição, e cada relação, com entidade de origem, entidade de destino, descrição e uma força de 1 a 10 (a Listagem 3.2 mostra o formato com um exemplo real deste trabalho).

Listagem 3.2 – Formato de saída da extração de entidades e relações (exemplo simplificado de uma resposta real sobre o art. 49 do CDC).

```
("entity"<|>ART. 49 DO CDC<|>dispositivo_legal<|>Dispositivo vigente ... Assegura ao consumidor
  o direito de desistir do contrato celebrado fora do estabelecimento comercial em sete dias ...)
##
("entity"<|>DIREITO DE ARREPENDIMENTO<|>direito<|>Permite ao consumidor desistir ...)
##
("relationship"<|>CONSUMIDOR<|>DIREITO DE ARREPENDIMENTO<|>O consumidor é titular do direito de
  arrependimento previsto no art. 49 ...<|>9)
##
<|COMPLETE|>
```

Para aumentar a cobertura, a extração pode ser repetida sobre a mesma unidade de texto em rodadas adicionais (*gleanings*), controladas por `max_gleanings`. Em cada rodada, a conversa continua com a mensagem fixa “MANY entities and relationships were missed in the last extraction” e o modelo acrescenta entidades e relações. Entre rodadas, o modelo é perguntado se ainda há entidades faltando (resposta “Y” ou “N”), o que pode encerrar o processo antes do limite. Com `max_gleanings` igual a 1, cada unidade de texto custa duas chamadas ao modelo.

Concluída a extração, os resultados de todas as unidades são combinados. Entidades com o mesmo nome **e** o mesmo tipo são fundidas em uma única entidade, que acumula a lista de descrições parciais e a lista de unidades de texto em que aparece (sua frequência). Relações com a mesma origem e o mesmo destino também são fundidas, e o peso resultante é a soma das forças atribuídas em cada ocorrência. Esse critério de fusão tem duas consequências práticas: nomes diferentes para a mesma entidade ("ART. 6", "ARTIGO 6º") geram nós duplicados, e o mesmo nome com tipos diferentes também gera nós distintos. Relações que citam entidades não extraídas são descartadas. Por isso, a padronização de nomes no *prompt* de extração tem efeito direto sobre a qualidade do grafo.

3.2.3 Resumo de descrições

Uma entidade que aparece em várias unidades de texto acumula várias descrições parciais. Na etapa de resumo (`summarize_descriptions`), que é executada dentro do *workflow* `extract_graph`, o LLM funde essas descrições em uma única descrição coerente, limitada a um número máximo de palavras (`max_length`). O mesmo é feito para as relações. Entidades e relações com uma única descrição não geram chamadas. Trata-se de uma tarefa simples de síntese, que pode ser atribuída a um modelo mais barato que o da extração.

3.2.4 Alegações (*claims*)

Opcionalmente, o *workflow* `extract_covariates` extrai alegações (*claims*): afirmações factuais associadas a entidades, com status (verdadeira, falsa, suspeita) e período. A etapa foi concebida para corpora como notícias e investigações e vem desabilitada por padrão. Neste trabalho ela não foi utilizada, pois no texto de uma lei as "afirmações" são as próprias normas, já representadas pelas entidades de tipo direito, obrigação e afins.

3.2.5 Detecção de comunidades

O *workflow* `finalize_graph` calcula atributos do grafo, como o grau de cada nó. Em seguida, `create_communities` aplica o algoritmo de Leiden hierárquico (Seção 2.5) ao grafo de entidades, usando o peso das relações. Comunidades com mais entidades que `max_cluster_size` (padrão: 10) são subdivididas recursivamente, formando os níveis da hierarquia. Por padrão, considera-se apenas o maior componente conexo do grafo (`use_lcc`), e uma semente fixa (`seed`) torna o particionamento determinístico.

3.2.6 Relatórios de comunidade

Para cada comunidade, de todos os níveis, o *workflow* `create_community_reports` monta um contexto com as entidades e relações da comunidade – e, nas comunidades de níveis superiores cujo conteúdo excede o limite `max_input_length`, com os relatórios das

subcomunidades – e pede ao LLM um relatório estruturado em JSON contendo: título, resumo, uma nota de importância de 0 a 10 com justificativa e uma lista de constatações (*findings*), cada uma com resumo e explicação. As afirmações do relatório citam as entidades e relações em que se baseiam. O relatório completo, convertido em texto, é o que as buscas global e DRIFT utilizam. O tamanho máximo do relatório é controlado por `max_length`. No método rápido, os relatórios são gerados a partir do texto original, com outro *prompt*.

3.2.7 *Embeddings*

Por fim, `generate_text_embeddings` gera, com o modelo de *embeddings*, vetores para três conjuntos de textos e os armazena no banco vetorial: as descrições das entidades (usadas pela busca local para encontrar as entidades relacionadas à pergunta), o conteúdo completo dos relatórios de comunidade (usado pela busca DRIFT) e o texto das unidades de texto (usado pela busca básica).

3.2.8 Cache

Todas as respostas dos modelos são gravadas em cache, indexadas pelo conteúdo da requisição (*prompt* e parâmetros). Se a indexação for executada novamente sem alteração de texto, *prompt* ou configuração de uma etapa, as respostas são lidas do cache, sem custo. Na prática, isso permite reexecutar a indexação – por exemplo, para alterar apenas a detecção de comunidades – pagando apenas pelas etapas que mudaram. Também permite que uma indexação interrompida seja retomada sem refazer as chamadas já concluídas.

3.3 Dados Produzidos

O resultado da indexação é um conjunto de tabelas em formato Parquet e um banco vetorial LanceDB, resumidos na Tabela 1. Cada registro tem um identificador interno (UUID) e um identificador curto sequencial (`human_readable_id`); é o identificador curto que aparece no contexto enviado ao modelo nas consultas e, portanto, nas citações das respostas (por exemplo, "`[Data: Sources (49)]`").

3.4 Modalidades de Consulta

O GraphRAG oferece quatro modalidades de consulta. Em todas, o resultado final é produzido por um LLM a partir de um *prompt* que contém instruções, o contexto recuperado formatado como tabelas e a pergunta, e as instruções pedem que cada afirmação cite os registros que a sustentam no formato `[Data: <tabela> (<ids>)]`. As modalidades diferem no que é recuperado, em quantas chamadas ao modelo são feitas e, portanto, no custo. Dois parâmetros são comuns às consultas: o **nível de comunidade** (`community_`

Tabela 1 – Tabelas produzidas pela indexação do GraphRAG 3.1.0.

Arquivo	**Conteúdo e principais colunas**
`documents.parquet`	Documentos de entrada: título, texto, metadados originais (`raw_data`) e unidades de texto do documento.
`text_units.parquet`	Unidades de texto: texto, número de *tokens*, documento de origem e ids das entidades e relações extraídas dela.
`entities.parquet`	Entidades: nome (`title`), tipo, descrição resumida, unidades de texto de origem, frequência e grau.
`relationships.parquet`	Relações: origem, destino, descrição, peso, grau combinado das extremidades e unidades de texto de origem.
`communities.parquet`	Comunidades: nível, comunidade-mãe e filhas, entidades, relações e unidades de texto que contém e tamanho.
`community_reports.parquet`	Relatórios: título, resumo, nota (`rank`) e sua justificativa, constatações, conteúdo completo em texto e JSON.
`covariates.parquet`	Alegações (apenas se `extract_claims` estiver habilitado).
`lancedb/`	Banco vetorial com os *embeddings* das descrições de entidades, dos relatórios e das unidades de texto.
`stats.json`, `context.json`	Estatísticas da execução (tempo por *workflow*) e estado do *pipeline*.

`level`, padrão 2 na CLI), que determina até que nível da hierarquia os relatórios de comunidade são considerados, e o **tipo de resposta** (`response_type`, padrão "*Multiple Paragraphs*"), que descreve ao modelo o formato desejado.

3.4.1 Busca local

A busca local é indicada para perguntas sobre entidades específicas. Seu ponto de partida é o grafo: a pergunta é convertida em *embedding* e comparada às descrições das entidades, e as entidades mais similares (`top_k_entities`, padrão 10) são selecionadas. A partir delas, o contexto é expandido com as relações dessas entidades (até `top_k_relationships` por entidade, priorizando as que ligam entidades selecionadas entre si), os relatórios das comunidades a que pertencem e as unidades de texto de onde foram extraídas.

Todo esse material compete por um orçamento de contexto (`max_context_tokens`, padrão 12 mil *tokens*), dividido em três partes por dois parâmetros: `text_unit_prop` (padrão 0,5) reserva uma fração para as unidades de texto, `community_prop` (padrão 0,15) reserva outra para os relatórios de comunidade, e o restante (0,35) é preenchido com as tabelas de entidades e relações. Cada parte é preenchida em ordem de relevância até esgotar sua fração. O contexto resultante tem a forma de tabelas delimitadas, com os identificadores curtos que o modelo deve citar. A busca local faz uma única chamada ao modelo.

3.4.2 Busca global

A busca global é indicada para perguntas sobre o corpus como um todo e segue a estratégia *map-reduce* do artigo original (EDGE et al., 2024):

1. são selecionados os relatórios de comunidade até o nível `community_level`;
2. **map**: os relatórios são embaralhados e agrupados em lotes de até `max_context_tokens` *tokens*; cada lote é enviado ao modelo em uma chamada separada, que devolve uma lista de pontos-chave relevantes para a pergunta, cada um com uma nota de importância de 0 a 100;
3. **reduce**: os pontos-chave de todos os lotes com nota maior que zero são ordenados pela nota e, até o limite `data_max_tokens`, enviados a uma chamada final que redige a resposta.

O número de chamadas é aproximadamente o número de lotes mais um, e cresce rapidamente com o nível de comunidade: níveis mais profundos têm mais comunidades e, portanto, mais relatórios. Como alternativa ao nível fixo, a seleção dinâmica de comunidades percorre a hierarquia a partir da raiz e pergunta ao modelo, para cada relatório visitado, se ele é relevante para a pergunta, descendo apenas pelos ramos relevantes; essa seleção faz chamadas adicionais de classificação.

3.4.3 Busca DRIFT

A busca DRIFT (*Dynamic Reasoning and Inference with Flexible Traversal*) (RESEARCH, 2024) combina as buscas global e local de forma iterativa, conforme a Figura 3. No código da versão 3.1.0, ela funciona da seguinte forma:

1. **Expansão da pergunta**: o modelo escreve uma resposta hipotética à pergunta, usando como modelo de formato um relatório de comunidade sorteado. O *embedding* dessa resposta é usado para a busca, segundo a técnica HyDE (GAO et al., 2023a) (1 chamada);
2. **Seleção de relatórios**: os `drift_k_followups` relatórios (padrão 20) mais similares à resposta hipotética são selecionados;
3. **Primer**: os relatórios selecionados são divididos em `primer_folds` grupos (padrão 5), e cada grupo é enviado ao modelo, que devolve uma resposta parcial, uma nota e uma lista de perguntas de acompanhamento (*follow-ups*);
4. **Aprofundamento**: em cada uma de `n_depth` rodadas (padrão 3), até `drift_k_followups` perguntas ainda não respondidas são escolhidas, e cada uma é respondida

por uma busca local completa, que também gera novas perguntas de acompanhamento;

5. **Redução**: todas as respostas parciais são enviadas a uma chamada final, que redige a resposta.

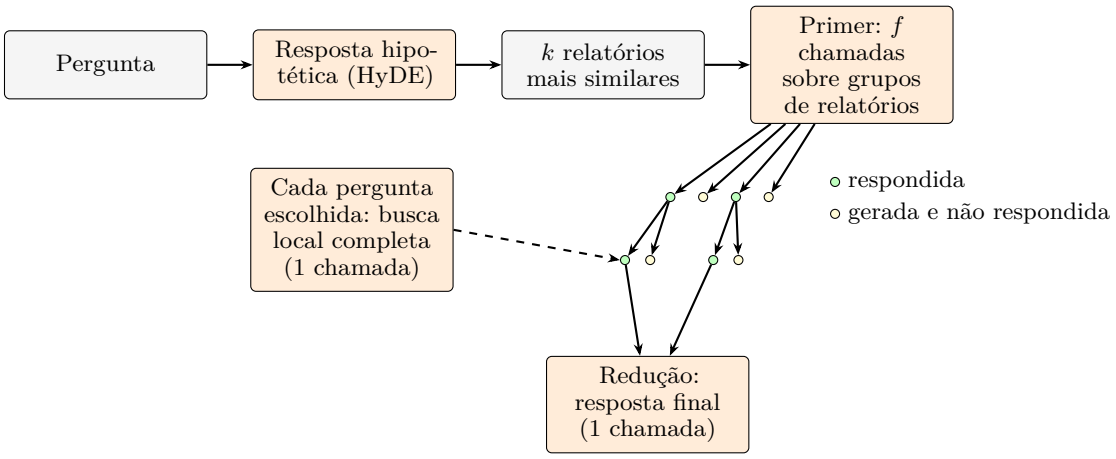


Figura 3 – Funcionamento da busca DRIFT no GraphRAG 3.1.0: a cada rodada, perguntas de acompanhamento são respondidas por buscas locais e geram novas perguntas.

Dois comportamentos do código, não descritos na documentação, são relevantes para a interpretação dos resultados. Primeiro, as perguntas aprofundadas em cada rodada são escolhidas **aleatoriamente** entre as não respondidas: a função que as ordena é chamada sem critério de pontuação e, nesse caso, embaralha a lista. As notas atribuídas pelo modelo às perguntas não são usadas para priorizá-las. Segundo, como cada busca local gera novas perguntas, a árvore final contém muitas perguntas que foram geradas e nunca respondidas, o que é o funcionamento normal do algoritmo. Com os valores padrão, o número de chamadas é $1 + 5 + 3 \times 20 + 1 = 67$, valor confirmado em uma consulta real deste trabalho (Capítulo 5).

3.4.4 Busca básica

A busca básica (*basic search*) é uma implementação de RAG vetorial tradicional: a pergunta é convertida em *embedding*, as k unidades de texto mais similares (padrão 10) são recuperadas do banco vetorial e inseridas no *prompt*, e o modelo gera a resposta em uma única chamada. Ela não utiliza entidades, relações ou comunidades. Por compartilhar com as demais modalidades o mesmo corpus, a mesma segmentação, o mesmo modelo de *embeddings* e o mesmo modelo gerador, ela é usada neste trabalho como *baseline*: a diferença entre ela e as demais modalidades é, essencialmente, o uso da estrutura de grafo.

3.4.5 Comparação entre as modalidades

A Tabela 2 resume as quatro modalidades.

Tabela 2 – Comparação entre as modalidades de consulta do GraphRAG 3.1.0 (valores padrão).

Modalidade	**Ponto de partida**	**Contexto enviado ao modelo**	**Chamadas ao modelo**
Básica	Unidades de texto similares à pergunta	10 unidades de texto	1
Local	Entidades similares à pergunta	Entidades, relações, relatórios e unidades de texto (12 mil *tokens*)	1
Global	Todos os relatórios até o nível escolhido	Lotes de relatórios (map); pontos-chave (reduce)	Número de lotes + 1; depende do nível
DRIFT	Relatórios similares a uma resposta hipotética	Relatórios (primer); contexto da busca local (cada pergunta); respostas parciais (redução)	$1 + f + d \times k + 1$ (f grupos, d rodadas, k perguntas por rodada)

3.5 *Prompts*

Todas as interações do GraphRAG com os modelos são guiadas por *prompts* armazenados em arquivos de texto na pasta `prompts`, cujos caminhos são definidos no `settings.yaml`. Os *prompts* contêm marcadores entre chaves (por exemplo, `{input_text}` e `{context_data}`) que a biblioteca substitui pelo conteúdo de cada chamada. A Tabela 3 relaciona os *prompts*, a etapa que os utiliza e os marcadores de cada um. Os *prompts* de indexação determinam o que entra no grafo; alterá-los exige reindexar. Os de consulta determinam como as respostas são redigidas e podem ser alterados sem reindexação.

O *prompt* de extração é o mais importante, por definir o grafo. Ele é organizado em seções: objetivo ("*-Goal-*"), passos ("*-Steps-*"), com a lista de tipos de entidade e o formato de saída; exemplos ("*-Examples-*"), cada um com um texto de entrada e a extração esperada; e os dados reais ("*-Real Data-*"), onde a unidade de texto é inserida. Os exemplos ensinam ao modelo, por demonstração, o nível de detalhe esperado, a forma de nomear entidades e o idioma das descrições. Como o *prompt* é enviado em cada chamada de extração, seu tamanho multiplica o custo da indexação: um *prompt* de 12 mil *tokens* enviado em 260 chamadas representa mais de 3 milhões de *tokens* de entrada.

3.5.1 Ajuste automático de *prompts*

O comando `graphrag prompt-tune` gera *prompts* de indexação adaptados ao corpus. Ele seleciona uma amostra de unidades de texto (por sorteio, pelas primeiras, por todas ou por similaridade) e, com o LLM: (i) identifica o domínio do corpus; (ii) cria uma *persona* de especialista nesse domínio; (iii) descobre os tipos de entidade presentes, ou usa nenhum tipo pré-definido; (iv) gera exemplos de extração a partir das unidades amostradas; (v) define o papel do redator dos relatórios de comunidade e o critério da nota

Tabela 3 – *Prompts* do GraphRAG 3.1.0.

Arquivo	**Etapa**	**Marcadores**
Indexação		
`extract_graph.txt`	Extração de entidades e relações	`input_text`
`summarize_descriptions.txt`	Resumo de descrições	`entity_name`, `description_list`
`extract_claims.txt`	Extração de alegações	`input_text`, `entity_specs`, `claim_description`
`community_report_graph.txt`	Relatórios (método padrão)	`input_text`
`community_report_text.txt`	Relatórios (método rápido)	`input_text`, `max_report_length`
Consulta		
`local_search_system_prompt.txt`	Busca local	`context_data`, `response_type`
`global_search_map_system_prompt.txt`	Busca global (map)	`context_data`, `max_length`
`global_search_reduce_system_prompt.txt`	Busca global (reduce)	`report_data`, `response_type`, `max_length`
`global_search_knowledge_system_prompt.txt`	Busca global (conhecimento geral)	–
`drift_search_system_prompt.txt`	DRIFT (buscas locais)	`context_data`, `response_type`, `global_query`, `followups`
`drift_reduce_prompt.txt`	DRIFT (redução)	`context_data`, `response_type`
`basic_search_system_prompt.txt`	Busca básica	`context_data`, `response_type`
`question_gen_system_prompt.txt`	Geração de perguntas	`context_data`, `question_count`

de importância; e (vi) monta os *prompts* de extração, de resumo e de relatório, traduzindo as instruções de saída para o idioma do corpus. As limitações encontradas nesse comando, que levaram ao desenvolvimento de um *script* próprio, são descritas na Seção 3.7 e no Capítulo 4.

3.6 Configuração

O `settings.yaml` reúne toda a configuração do GraphRAG em seções. Antes de interpretar o arquivo, a biblioteca substitui referências a variáveis de ambiente no formato `${NOME}` pelo valor correspondente, lido do arquivo `.env` da pasta raiz; é assim que as chaves de API são fornecidas sem que fiquem escritas na configuração. A substituição é feita no texto do arquivo inteiro, inclusive nos comentários (Seção 3.7). A configuração completa usada neste trabalho está no Apêndice A.

3.6.1 Modelos

As seções `completion_models` e `embedding_models` definem modelos nomeados, que são referenciados pelas demais seções por meio de identificadores (`completion_model_id` e `embedding_model_id`). Isso permite atribuir um modelo diferente a cada etapa da indexação e a cada modalidade de consulta. Para cada modelo, configuram-se: o provedor (`model_provider`, repassado ao LiteLLM) e o nome do modelo; a autenticação (chave de API ou identidade gerenciada do Azure); o número de requisições simultâneas (`concurrent_requests`) e o modo assíncrono; a política de novas tentativas em caso de erro (`max_retries` e estratégia, como espera exponencial); limites de taxa; e os argumentos repassados a cada chamada (`call_args`), como temperatura e tempo máximo de espera.

3.6.2 Entrada, armazenamento e cache

As seções `input`, `input_storage`, `output_storage`, `cache`, `reporting` e `vector_store` definem o formato da entrada (texto, CSV ou JSON, e os campos de identificador, título e texto), onde ficam os arquivos de entrada e saída (sistema de arquivos local, Azure Blob Storage ou Cosmos DB), o cache, os registros e o banco vetorial (LanceDB, Azure AI Search ou Cosmos DB). A seção `snapshots` permite gravar artefatos intermediários adicionais, como o grafo em formato GraphML.

3.6.3 Indexação

A Tabela 4 apresenta as principais opções das etapas de indexação, com seus valores padrão e seus efeitos.

Tabela 4 – Principais opções de configuração da indexação.

Seção	**Opção**	**Padrão**	**Efeito**
`chunking`	`size`, `overlap`	1200, 100	Tamanho e sobreposição das unidades de texto. Unidades menores geram mais chamadas de extração e extrações mais detalhadas; maiores reduzem o custo e podem perder entidades.
	`prepend_metadata`	–	Campos de metadados inseridos no início de cada unidade, dando ao modelo o contexto do documento.
	`encoding_model`	`o200k_base`	Codificação usada para contar *tokens*.
`extract_graph`	`entity_types`	*organization, person, geo, event*	Tipos de entidade inseridos no *prompt* de extração. Definem a ontologia do grafo.

(continuação)

Seção	Opção	Padrão	Efeito
	`max_gleanings`	1	Rodadas adicionais de extração por unidade de texto. Cada rodada é uma chamada a mais por unidade.
	`prompt`, modelo	–	*Prompt* e modelo da extração, a etapa que mais influencia a qualidade e o custo.
`summarize_descriptions`	`max_length`	500	Tamanho máximo (em palavras) das descrições resumidas.
	`max_input_tokens`	4000	Limite de descrições parciais por chamada de resumo.
`cluster_graph`	`max_cluster_size`	10	Tamanho máximo de uma comunidade antes de ser subdividida. Valores menores geram mais níveis e mais comunidades, portanto mais relatórios.
	`use_lcc`, `seed`	sim, fixo	Uso apenas do maior componente conexo; semente do algoritmo.
`community_reports`	`max_length`	2000	Tamanho máximo dos relatórios. Relatórios maiores encarecem a indexação e todas as consultas global e DRIFT.
	`max_input_length`	8000	Tamanho máximo do contexto de cada relatório.
`extract_claims`	`enabled`	não	Habilita a extração de alegações (mais uma chamada por unidade de texto).
`extract_graph_nlp`, `prune_graph`	diversas	–	Extração por PLN tradicional e poda do grafo, usadas no método rápido.
`embed_text`	`names`, `batch_size`	3 conjuntos, 16	Textos que recebem *embeddings* e tamanho dos lotes enviados ao modelo.
(raiz)	`workflows`	–	Lista explícita de *workflows*, substituindo a do método escolhido.

3.6.4 Consultas

As opções das modalidades de consulta estão nas seções `local_search`, `global_search`, `drift_search` e `basic_search`. Além do modelo e dos *prompts*, elas controlam o tamanho do contexto e, nas modalidades com várias chamadas, quantas chamadas são feitas. A Tabela 5 apresenta as opções e seus efeitos esperados em custo, com os valores medidos neste trabalho quando disponíveis.

Tabela 5 – Opções de configuração das modalidades de consulta e seus efeitos.

Seção	**Opção**	**Padrão**	**Efeito**
`basic_search`	`k`	10	Número de unidades de texto recuperadas. Mais unidades aumentam a cobertura e os *tokens* de entrada.
	`max_context_tokens`	12000	Teto do contexto.
`local_search`	`top_k_entities`	10	Entidades selecionadas a partir da pergunta: a porta de entrada no grafo.
	`top_k_relationships`	10	Relações por entidade selecionada.
	`text_unit_prop`	0,5	Fração do contexto para unidades de texto.
	`community_prop`	0,15	Fração para relatórios de comunidade. O restante vai para entidades e relações.
	`max_context_tokens`	12000	Teto do contexto: é o principal determinante do custo da busca local.
	`conversation_history_max_turns`	5	Turnos de conversa anteriores incluídos no contexto.
`global_search`	`community_level` (consulta)	2	Nível da hierarquia: define quantos relatórios são lidos e, portanto, o número de chamadas map. É o principal determinante do custo.
	`max_context_tokens`	12000	Tamanho de cada lote do map. Lotes maiores significam menos chamadas, com o mesmo total de *tokens*.
	`data_max_tokens`	12000	Teto dos pontos-chave enviados ao reduce.
	`map_max_length`, `reduce_max_length`	1000, 2000	Tamanho pedido (em palavras) às respostas do map e do reduce.
	`dynamic_search_*`	–	Limiar, profundidade e repetições da seleção dinâmica de comunidades, que faz chamadas extras de classificação.
`drift_search`	`drift_k_followups`	20	Número de relatórios do primer **e** de perguntas respondidas por rodada. Principal determinante do custo.
	`n_depth`	3	Número de rodadas de aprofundamento. Multiplica as chamadas.
	`primer_folds`	5	Número de grupos de relatórios no primer.
	`local_search_text_unit_prop`	0,9	Fração do contexto de cada busca local para unidades de texto.

(continuação)

Seção	Opção	Padrão	Efeito
	`local_search_community_prop`	0,1	Fração para relatórios. Com os padrões, sobra 0% para entidades e relações (Seção 3.7).
	`local_search_top_k_mapped_entities`, `_top_k_relationships`	10, 10	Iguais aos da busca local.
	`local_search_max_data_tokens`	12000	Teto do contexto de cada busca local; multiplicado pelo número de perguntas respondidas.
	`local_search_temperature`, `reduce_temperature`	0	Únicas temperaturas definidas por padrão nas consultas (Seção 3.7).

Do ponto de vista de custo, as opções se dividem em dois grupos. As que controlam o **tamanho do contexto** (`max_context_tokens`, `k`, proporções) alteram os *tokens* de cada chamada de forma aproximadamente linear. As que controlam o **número de chamadas** (`community_level` na busca global; `drift_k_followups` e `n_depth` na DRIFT) multiplicam o custo e são as que mais pesam: a busca global no nível 2 e a DRIFT com os valores padrão custam, cada uma, mais de cinquenta vezes o que custa uma busca local. Uma análise detalhada de cada opção, com cenários de custo estimados a partir de medições, foi registrada na documentação do projeto e fundamenta as escolhas de configuração da avaliação.

3.7 Defeitos e Comportamentos Não Documentados

O uso do GraphRAG fora dos exemplos da documentação – em português, com taxonomia própria, com modelos de raciocínio e chamado como biblioteca pelo *backend* – revelou defeitos e comportamentos não documentados da versão 3.1.0. Eles são relevantes porque vários afetam silenciosamente o resultado das consultas e, portanto, qualquer comparação entre modalidades. A Tabela 6 os resume; o tratamento dado a cada um é descrito nos capítulos 4 e 5.

Tabela 6 – Defeitos e comportamentos não documentados identificados no GraphRAG 3.1.0.

#	**Problema**	**Efeito**
1	O método `search()` da busca básica chama a geração em modo *streaming* sem aguardar a corrotina (`await` ausente).	O erro é capturado internamente e a busca básica **sempre devolve resposta vazia**, sem aviso, quando usada como biblioteca. A CLI não é afetada por usar outro método. O defeito persiste na versão 3.2.0. Sem correção, o *baseline* da comparação seria inválido.
2	O ajuste automático de *prompts* reutiliza a mesma lista de mensagens em todas as chamadas de geração de exemplos.	Cada chamada recebe os textos de todos os exemplos, e o modelo responde sempre sobre o último: todos os exemplos de extração saem do mesmo trecho.
3	O ajuste automático de *prompts* não aceita uma taxonomia definida pelo usuário.	Os tipos "descobertos" pelo modelo são gravados fixos no *prompt*, ignorando os `entity_types` da configuração; sem a descoberta, o *prompt* sai sem tipo algum. O comando exige ainda um modelo com identificador fixo (`default_completion_model`).
4	A leitura dos *prompts* de consulta resolve o caminho em relação ao diretório corrente e, se o arquivo não existir, usa o *prompt* padrão.	Um erro no nome ou no caminho do arquivo faz a consulta usar silenciosamente o *prompt* padrão em inglês.
5	A substituição de variáveis de ambiente é aplicada ao `settings.yaml` inteiro, incluindo comentários.	Um cifrão em um comentário ou uma referência comentada a uma variável inexistente impede o carregamento da configuração.
6	Apenas a DRIFT define temperatura nas consultas, com valor padrão 0, herança da versão 2.	Inconsistência entre modalidades; com modelos de raciocínio, que só aceitam temperatura 1, a DRIFT falha.
7	Na DRIFT, as proporções padrão das buscas locais somam 1 $(0{,}9 + 0{,}1)$.	Não sobra espaço para entidades e relações; e, neste índice, nenhum relatório cabe nos 10% restantes. As buscas locais da DRIFT passam a usar apenas unidades de texto, ou seja, o grafo só é usado no primer.
8	Na DRIFT, as perguntas de acompanhamento de cada rodada são escolhidas por sorteio.	Fonte adicional de variação entre execuções; as notas das perguntas não influenciam a exploração.
9	Nas buscas local e básica, o contexto recuperado só é notificado aos *callbacks* depois da geração da resposta.	Não é possível exibir o contexto antes da resposta usando apenas a interface de *callbacks*.
10	As métricas de *tokens* das consultas são calculadas pela contagem do texto enviado e recebido.	Não incluem os *tokens* de raciocínio dos modelos de raciocínio e, portanto, subestimam o custo real.

(continuação)

#	**Problema**	**Efeito**
11	A versão do LiteLLM utilizada não reconhece modelos lançados depois dela.	Parâmetros não suportados pelo modelo não são filtrados automaticamente e precisam ser definidos de forma compatível na configuração.

4 Indexação do Código de Defesa do Consumidor

Este capítulo descreve como o GraphRAG foi aplicado ao CDC: da obtenção e estruturação do texto da lei à análise do grafo de conhecimento gerado. A Seção 4.1 apresenta a primeira versão do projeto, executada com modelos locais, e os motivos da mudança para modelos em nuvem. As seções 4.2 e 4.3 tratam da preparação do corpus. As seções 4.4 a 4.7 descrevem as decisões de configuração: modelos, temperatura, taxonomia e *prompts*. As seções 4.9 a 4.12 apresentam a execução da indexação, o grafo resultante, o custo e a publicação do índice.

4.1 Primeira Versão: Modelos Locais

A primeira versão da prova de conceito foi executada inteiramente na máquina da autora, com o objetivo de evitar custos de API. Os modelos eram servidos pelo Ollama (OLLAMA, 2026) em um contêiner: o `llama3.1:8b`, com janela de contexto ampliada para 8.192 *tokens*, para geração, e o `nomic-embed-text` para *embeddings*. Como a máquina não dispõe de GPU, a configuração limitava o GraphRAG a uma requisição simultânea, com tempo máximo de 30 minutos por chamada.

Essa versão permitiu montar o ambiente em contêineres, entender o *pipeline* e construir o primeiro visualizador do grafo, mas mostrou-se inviável para o trabalho por três motivos. Primeiro, o tempo: cada chamada ao modelo levava em torno de sete minutos, e a indexação completa, horas. Segundo, a qualidade: o modelo pequeno produzia extrações incompletas e inconsistentes, com tipos de entidade e nomes fora do padrão. Terceiro, a configuração: os *prompts*, gerados pelo ajuste automático com o próprio modelo local, usavam tipos de entidade em inglês descobertos pelo modelo (*law*, *consumer*, *business*, *government*, *product*, *service*), e não a taxonomia definida na configuração, e a pasta de entrada continha, além do texto da lei, uma versão antiga e incompleta do CDC, de modo que dois documentos sobrepostos eram indexados. A configuração local foi preservada como alternativa no repositório, idêntica à de nuvem exceto pelos modelos, e o trabalho seguiu com modelos em nuvem.

4.2 Preparação do Corpus

4.2.1 Fonte e estratégia

O texto do CDC foi obtido da versão compilada publicada no portal da legislação federal (BRASIL, 1990), que incorpora as alterações posteriores e mantém os dispositivos vetados e revogados com a respectiva indicação. Em vez de indexar o texto bruto – o que deixaria a segmentação cortar artigos ao meio e perderia a hierarquia –, optou-se por estruturar o texto em **um registro por artigo**. Essa escolha segue a própria técnica legislativa, que define o artigo como unidade básica da lei (Seção 2.7.1), e faz com que cada unidade de texto do GraphRAG corresponda a um artigo completo.

4.2.2 O *parser* da legislação

Foi desenvolvido em Python um *parser* que lê o texto bruto e reconhece, por expressões regulares, os níveis da hierarquia da Lei Complementar nº 95/1998 (BRASIL, 1998) (parte, livro, título, capítulo, seção e subseção) e o início de cada artigo. Para cada artigo, gera um registro com a identificação da lei, o número e o nome de cada nível da hierarquia em que o artigo se encontra, a situação normativa e o texto completo (*caput*, parágrafos, incisos e alíneas). O *parser* foi escrito de forma genérica, para poder ser reaproveitado em outras leis federais. A Listagem 4.1 mostra um registro.

Listagem 4.1 – Registro de um artigo no arquivo `cdc.json` (campos nulos omitidos; texto abreviado).

```
{
  "id": "cdc_art_49",
  "artigo_num": "49",
  "lei_sigla": "CDC",
  "lei_nome": "Código de Defesa do Consumidor",
  "lei_numero": "Lei nº 8.078, de 11 de setembro de 1990",
  "titulo_num": "I",   "titulo_nome": "Dos Direitos do Consumidor",
  "capitulo_num": "VI", "capitulo_nome": "Da Proteção Contratual",
  "secao_num": "I",    "secao_nome": "Disposições Gerais",
  "situacao": "vigente",
  "dispositivos_vetados_ou_revogados": [],
  "text": "Art. 49. O consumidor pode desistir do contrato, no prazo de 7 dias
      ..."
}
```

A primeira versão do *parser*, herdada da fase inicial do projeto, apresentava problemas que foram corrigidos antes da indexação (Tabela 7). O mais grave era a perda do sufixo de letra dos artigos acrescentados por leis posteriores, que fazia os artigos 54 e 54-A a 54-G receberem o mesmo identificador. Como o GraphRAG usa o identificador do documento, registros com identificadores repetidos se confundiriam. O *parser* passou a falhar caso encontre identificadores duplicados.

Tabela 7 – Problemas corrigidos no *parser* da legislação.

Problema	**Causa**	**Correção**
Identificadores repetidos (arts. 54 e 54-A a 54-G; 42-A; 104-A a 104-C)	A expressão regular do artigo descartava o sufixo de letra	O número do artigo inclui o sufixo e o identificador inclui a sigla da lei (`cdc_art_54-A`)
Cabeçalhos como "Art. 11º" e "Art. 54º-C"	O cabeçalho era remontado sempre com ordinal	Regra da LC nº 95/1998: ordinal até o nono artigo, cardinal a partir do décimo
Capítulo VI-A lido como "VI", com nome "-A"	A regra dos níveis da hierarquia não aceitava sufixo	Níveis com sufixo de letra; nome lido da linha seguinte
Artigos inteiros marcados como vetados quando apenas um parágrafo o era (ex.: arts. 6º e 37)	Busca da expressão "(Vetado)" em todo o texto do artigo	Novos campos de situação, descritos a seguir

4.2.3 Dispositivos vetados e revogados

Uma decisão importante foi **manter** no corpus todo o texto vetado ou revogado. Do ponto de vista jurídico, o registro de um veto também é informação: saber que o art. 54-E, que seria incluído pela Lei nº 14.181/2021 (BRASIL, 2021), foi vetado responde a uma pergunta legítima sobre a lei. Para que essa informação seja representada corretamente, cada registro recebe dois campos:

- `situacao`: `vigente`, `vetado` ou `revogado`, referente ao artigo como um todo;
- `dispositivos_vetados_ou_revogados`: os parágrafos, incisos ou alíneas vetados ou revogados dentro de um artigo vigente – por exemplo, para o art. 51, os incisos V e XIX e o § 3º.

O CDC compilado resultou em 130 artigos, dos quais 12 inteiramente vetados (arts. 11, 15, 16, 45, 54-E, 62, 85, 86, 89, 96, 108 e 109), e 30 dispositivos vetados distribuídos em 21 artigos vigentes. O *parser* oferece uma opção para gerar a versão sem os dispositivos vetados (118 artigos), o que permite, se desejado, comparar os dois cenários.

4.2.4 Configuração da entrada

O arquivo resultante (`cdc.json`) é lido pelo GraphRAG no formato JSON, com o campo `id` como identificador e título do documento e o campo `text` como conteúdo. Os demais campos ficam disponíveis como metadados. Os nomes dos campos foram mantidos em português de propósito, já que aparecem para o modelo no início de cada trecho, como descrito a seguir.

4.3 Segmentação

O tamanho das unidades de texto foi mantido no padrão de 1.200 *tokens*, com sobreposição de 100. Como o maior artigo do CDC tem 781 *tokens* (a média é de 182), cada artigo cabe inteiro em uma unidade de texto: o índice tem 130 unidades, uma por artigo, e nenhuma mistura artigos diferentes ou corta um artigo ao meio.

Para preservar a hierarquia da lei, foi utilizada a opção `prepend_metadata`, que insere no início de cada unidade de texto a sigla da lei, os nomes do título, do capítulo e da seção e a situação do artigo (Listagem 4.2). Com isso, o modelo sabe, ao extrair entidades do art. 39, que ele está no capítulo das práticas comerciais e na seção das práticas abusivas, e sabe, ao processar o art. 54-E, que ele foi vetado. A mesma informação aparece para o modelo nas consultas, sempre que uma unidade de texto é incluída no contexto.

Listagem 4.2 – Início de uma unidade de texto, com os metadados inseridos pelo `prepend_metadata`.

```
1 lei_sigla: CDC.
2 titulo_nome: Dos Direitos do Consumidor.
3 capitulo_nome: Da Proteção Contratual.
4 secao_nome: Disposições Gerais.
5 situacao: vigente.
6 Art. 49. O consumidor pode desistir do contrato, no prazo de 7 dias a contar de
      sua assinatura
7 ou do ato de recebimento do produto ou serviço, sempre que a contratação de
      fornecimento de
8 produtos e serviços ocorrer fora do estabelecimento comercial, especialmente por
      telefone ou
9 a domicílio. ...
```

4.4 Escolha dos Modelos

Com o corpus pequeno (cerca de 24 mil *tokens* nas 130 unidades de texto), o custo da indexação é dominado pelo número de chamadas e pelo tamanho dos *prompts*, e não pelo volume de texto. A estratégia adotada foi usar, em cada etapa, o modelo mais barato capaz de executá-la bem, conforme a Tabela 8. O GraphRAG permite essa combinação porque cada etapa referencia um modelo nomeado na configuração (Seção 3.6.1). Foram definidos dois modelos de geração – `strong_model` (`gpt-6.1-sol`) e `cheap_model` (`gpt-6-luna`) – e um de *embeddings*.

A OpenAI foi escolhida como provedor principal por três razões: o GraphRAG foi desenvolvido e testado pela Microsoft com modelos dessa família, o que torna a avaliação mais fiel ao funcionamento previsto da ferramenta; um único provedor fornece tanto os modelos de geração quanto os de *embeddings*, com uma única chave de API; e seus modelos de *embeddings* estão entre os de menor custo. Como o GraphRAG acessa os modelos pelo

Tabela 8 – Modelos utilizados em cada etapa.

Etapa	**Modelo**	**Justificativa**
Extração de entidades e relações	`gpt-6.1-sol`	Define a qualidade do grafo; é a etapa mais importante.
Resumo de descrições	`gpt-6-luna`	Tarefa simples: fundir descrições da mesma entidade.
Relatórios de comunidade	`gpt-6.1-sol`	Base das buscas global e DRIFT.
Respostas das consultas (todas as modalidades)	`gpt-6.1-sol`	O mesmo modelo gerador em todas as modalidades, para que a comparação dependa apenas da recuperação.
Embeddings	`text-embedding-3-large`	Vetores de 3.072 dimensões; custo desprezível para o tamanho do corpus.

LiteLLM, configurações equivalentes para outros provedores (Anthropic e Google) foram preparadas e mantidas no repositório, permitindo a troca sem alteração de código.

Antes da indexação, foi criado um *script* de teste que lê os modelos configurados e faz uma chamada mínima a cada um, de duas formas: como na indexação e com os parâmetros usados pela busca DRIFT. O teste custa frações de centavo e revelou, antes de qualquer gasto relevante, duas restrições dos modelos escolhidos: eles não aceitam o parâmetro `max_tokens`, apenas `max_completion_tokens` (que o GraphRAG 3 já utiliza), e só aceitam temperatura igual a 1. Revelou também que a versão do LiteLLM instalada com o GraphRAG não reconhece esses modelos e, por isso, não filtra automaticamente os parâmetros não suportados, que precisam ser definidos de forma compatível na configuração.

4.5 Temperatura

A restrição de temperatura motivou uma análise de como o GraphRAG trata esse parâmetro. Na versão 3, a configuração de modelos passou a usar argumentos genéricos de chamada, sem temperatura padrão; assim, a indexação e as buscas local, global e básica usam o padrão do provedor, que é 1. A busca DRIFT, porém, manteve campos próprios de temperatura com valor padrão 0 – herança da versão 2, em que todos os modelos usavam temperatura 0 por padrão. O resultado é uma inconsistência entre modalidades, e, com modelos que só aceitam temperatura 1, a DRIFT falharia com erro de parâmetro.

A decisão foi fixar explicitamente a temperatura 1 nos argumentos de chamada dos dois modelos de geração e nos dois campos da DRIFT. Assim, todas as etapas e todas as modalidades usam a mesma temperatura, e ela deixa de ser uma variável de confusão na comparação com o *baseline*. A consequência metodológica é que as respostas variam entre execuções da mesma pergunta. Por isso, a avaliação (Capítulo 6) foi planejada sobre um índice congelado, igual para todas as modalidades, e com repetições de cada pergunta, o que permite medir também a consistência das respostas. Como se verá, a temperatura não é a única fonte de variação: a busca global embaralha os relatórios antes de formar

os lotes, e a DRIFT sorteia o modelo de formato da resposta hipotética e as perguntas aprofundadas.

4.6 Taxonomia de Entidades

Os tipos de entidade definem a ontologia do grafo: o modelo só extrai entidades dos tipos listados. A taxonomia inicial do projeto tinha oito tipos (artigo, direito, obrigação, sanção, órgão fiscalizador, relação de consumo, fornecedor e consumidor) e apresentava problemas: faltavam conceitos centrais do CDC, como produto, serviço, contrato, práticas abusivas, crimes e ações coletivas; "órgão fiscalizador" não abrangia o Ministério Público, a Defensoria Pública, o Poder Judiciário e as associações; "relação de consumo" é um conceito, e não uma categoria de entidades; "sanção" misturava sanções administrativas e penais; e "artigo" não contemplava remissões a outras normas.

A taxonomia foi redefinida a partir de uma análise dos seis títulos do CDC, resultando nos catorze tipos da Tabela 9, agrupados por papel. Um tipo próprio para princípios chegou a ser proposto, mas foi incorporado a `instituto_juridico`, já que princípios e institutos são tratados de forma semelhante na lei e a separação aumentaria a indecisão do modelo.

Algumas decisões de projeto acompanharam a taxonomia:

- **Número de tipos.** Tipos demais fragmentam o grafo e deixam o modelo indeciso entre categorias próximas; catorze tipos cobrem os seis títulos do CDC com pouca sobreposição.
- **Prazos não são entidades.** Um prazo como "30 dias" aparece no CDC com sentidos diferentes (decadência do direito de reclamar por vícios aparentes, prazo para o fornecedor sanar o vício). Como o GraphRAG funde entidades de mesmo nome e tipo (Seção 3.2.2), uma entidade "30 DIAS" misturaria esses sentidos. Os prazos são registrados na descrição do direito, da obrigação ou do instituto a que se referem.
- **Definições dos tipos no *prompt*.** O GraphRAG insere no *prompt* apenas os nomes dos tipos. Como nomes como `instituto_juridico` são ambíguos para o modelo, uma definição de cada tipo foi acrescentada ao *prompt* de extração (Seção 4.7).
- **Regras de nomeação.** Pelo mesmo motivo – a fusão por nome –, foram definidas regras para padronizar os nomes das entidades: maiúsculas, singular, sem artigo inicial; termos do próprio CDC, sem sinônimos (FORNECEDOR, e não EMPRESA); artigos do CDC no padrão "ART. 54-A DO CDC", com o artigo como unidade (parágrafos e incisos vão na descrição); outras normas pelo nome usual ou número e ano ("LEI Nº 7.347/1985"); e artigos vetados ou revogados como entidades, com o

Tabela 9 – Taxonomia de tipos de entidade utilizada.

Grupo	**Tipo**	**Abrangência**
Sujeitos	`consumidor`	Consumidor e equiparados; consumidor superendividado
	`fornecedor`	Fabricante, produtor, construtor, importador, comerciante, prestador de serviço, fornecedor de crédito
	`orgao_entidade`	Procon, SNDC, Ministério Público, Defensoria Pública, entes federativos, Poder Judiciário, associações
Objeto	`produto`	Bens móveis ou imóveis, duráveis ou não, perigosos, peças de reposição
	`servico`	Serviços em geral, inclusive bancários, de crédito, securitários e públicos
	`contrato`	Contratos de consumo e de adesão, cláusulas, garantias, crédito, compra a prazo
Normativo	`direito`	Direitos do consumidor
	`obrigacao`	Deveres do fornecedor e do poder público
	`pratica_abusiva`	Práticas abusivas, publicidade enganosa ou abusiva, cobrança vexatória, cláusulas abusivas
	`instituto_juridico`	Princípios (vulnerabilidade, boa-fé) e conceitos (relação de consumo, responsabilidade objetiva, fato e vício, decadência e prescrição, inversão do ônus da prova, superendividamento)
Consequências e tutela	`sancao_administrativa`	Multa, apreensão, cassação, interdição, contrapropaganda
	`infracao_penal`	Crimes contra as relações de consumo e suas penas
	`instrumento_processual`	Ações coletivas, ação civil pública, interesses difusos, coletivos e individuais homogêneos, coisa julgada
Remissões	`dispositivo_legal`	Artigos do CDC e de outras normas citadas

veto registrado na descrição.

4.7 Ajuste dos *Prompts* de Indexação

4.7.1 Limitações do ajuste automático

O comando `graphrag prompt-tune` (Seção 3.5.1) não pôde ser usado diretamente, por quatro limitações identificadas na versão 3.1.0:

1. **Não aceita taxonomia.** Com a descoberta de tipos ativada (padrão), os tipos descobertos pelo modelo são gravados fixos no *prompt*, ignorando os definidos na configuração – foi o que gerou os tipos em inglês da versão local. Com a descoberta desativada, o *prompt* é gerado sem tipo algum.

2. **Exige um modelo com identificador fixo**, `default_completion_model`, incompatível com a configuração de dois modelos nomeados.

3. **Seleciona os trechos de exemplo por sorteio**, o que não é reprodutível.

4. **Todos os exemplos saem do mesmo trecho.** A função que gera os exemplos reutiliza o mesmo objeto de construção de mensagens em todas as chamadas. Como as chamadas só são executadas ao final, de forma concorrente, cada uma recebe os textos de todos os exemplos, e o modelo responde sempre sobre o último. Na primeira tentativa, os textos de entrada dos exemplos eram os dos arts. 6º, 12, 51 e 56, mas as quatro extrações geradas eram do art. 82. O defeito foi confirmado com um teste mínimo, em que cada chamada recebia quatro mensagens em vez de duas.

4.7.2 *Script* próprio de ajuste

Foi desenvolvido um *script* que executa as mesmas etapas e utiliza as mesmas funções do ajuste automático da biblioteca, com as seguintes diferenças:

- usa a taxonomia da configuração, e falha se a lista de definições do *script* divergir dela;
- insere no *prompt* de extração, antes da seção de exemplos, as definições de cada tipo e as regras de nomeação (Listagem 4.3), e fornece as mesmas orientações como mensagem de sistema na geração dos exemplos, para que estes já sigam as regras;
- fixa o domínio ("legislação consumerista brasileira (Código de Defesa do Consumidor, Lei nº 8.078/1990)") e o idioma ("português do Brasil"), em vez de inferi-los;
- gera cada exemplo em uma conversa independente, contornando o defeito descrito acima;
- monta os trechos de exemplo exatamente como na indexação, com o mesmo segmentador e os mesmos metadados;
- normaliza os exemplos gerados (nomes em maiúsculas, correção de tipos com erro de digitação, como `instrument_processual`) e alerta se um exemplo não cita o próprio artigo;
- registra em um arquivo de metadados o modelo, a versão do GraphRAG, a *persona*, o papel do redator dos relatórios, o critério de nota, os artigos usados e os alertas, para fins de reprodutibilidade.

Listagem 4.3 – Trecho do *prompt* de extração com as seções acrescentadas pelo *script* de ajuste (abreviado).

```
1 -Definições dos tipos de entidade-
2 - consumidor: consumidor e equiparados (coletividade de pessoas, vítimas do
      evento, pessoas
3   expostas às práticas comerciais), incluindo o consumidor superendividado.
```

```
4 - fornecedor: quem fornece produtos ou serviços: fabricante, produtor, construtor
    , ...
5 - instituto_juridico: princípios e conceitos jurídicos: princípios da Política
    Nacional das
6   Relações de Consumo (vulnerabilidade, boa-fé, harmonização), relação de consumo
      , ...
7   ...
8 -Regras de nomeação das entidades-
9 - Escreva os nomes das entidades em MAIÚSCULAS, no singular, em português e sem
    artigo
10  inicial (CONSUMIDOR, e não O CONSUMIDOR).
11 - Use os termos do próprio CDC, sem sinônimos (FORNECEDOR, e não EMPRESA; PRODUTO
    , e não
12  MERCADORIA).
13 - Nomeie artigos do CDC como ART. 6º DO CDC, ART. 54-A DO CDC ou ART. 10 DO CDC
    ...
14 - Prazos não são entidades: registre-os na descrição do direito, da obrigação ou
    do
15  instituto a que se referem ...
16 - Se o artigo estiver vetado ou revogado (situacao no cabeçalho do trecho), crie
    a entidade
17  do dispositivo e registre o veto ou a revogação na descrição.
18 -Examples-
19 ...
```

4.7.3 Escolha dos exemplos

A escolha dos artigos usados como exemplos envolve um equilíbrio entre qualidade e custo. Na primeira tentativa, com os arts. 6º, 12, 51, 56 e 82, os exemplos eram corretos, mas os arts. 6º e 51 – os mais longos e densos do código – geraram extrações com 50 entidades e 113 relações e com 37 entidades e 72 relações, respectivamente. O *prompt* de extração resultante tinha cerca de 27 mil *tokens*, que seriam enviados em cada uma das centenas de chamadas de extração.

Na versão final, foram escolhidos artigos curtos que, em conjunto, cobrem a maior parte da taxonomia, em ordem de prioridade: 49 (direito de arrependimento), 12 (responsabilidade pelo fato do produto), 42 (cobrança de dívidas), 82 (legitimados para ações coletivas), 66 (crime de afirmação falsa ou enganosa) e 56 (sanções administrativas). Os exemplos entram no *prompt* nessa ordem enquanto o total couber em 12 mil *tokens*, com mínimo de três. Resultaram os exemplos dos arts. 49, 12 e 42 e um *prompt* de extração de 11.709 *tokens*. Os tipos sem exemplo (órgãos, instrumentos processuais, sanções e crimes) são orientados apenas pelas definições. Para a geração da *persona* e do papel do redator dos relatórios de comunidade, foram usados como contexto, além dos exemplos, artigos representativos de todo o código (arts. 2º, 4º, 6º, 18, 26, 39, 51, 54-A, 54-E e 106).

Os *prompts* de indexação gerados combinam instruções em inglês (as do GraphRAG) com conteúdo em português (definições, regras, exemplos e o papel do redator), e determinam que as descrições sejam escritas em português do Brasil. O *prompt* de relatórios

de comunidade, por exemplo, define o redator como "um analista jurídico especializado em legislação consumerista brasileira" e a nota de importância como a relevância do conteúdo para a compreensão da proteção jurídica do consumidor. Os *prompts* de consulta foram mantidos os padrões da biblioteca.

4.8 Configuração Final

A configuração completa está no Apêndice A. Além das escolhas já descritas, a preparação da configuração revelou erros que teriam impedido a indexação ou alterado silenciosamente o resultado (Tabela 10). Dois deles decorrem da substituição de variáveis de ambiente em todo o arquivo (Seção 3.7); por isso, um aviso foi colocado no topo do `settings.yaml`.

Tabela 10 – Erros de configuração encontrados e corrigidos antes da indexação.

Erro	**Efeito**	**Correção**
Caminho do banco vetorial com barra invertida (`output\lancedb`)	No contêiner Linux, criava uma pasta com a barra no nome	Barra normal
Nome inexistente para o *prompt* de redução da DRIFT	A DRIFT usaria silenciosamente o *prompt* padrão	Nome corrigido
Cifrão no padrão de nomes de arquivo de entrada	Erro de interpretação da configuração	Linha removida (o padrão da biblioteca é o mesmo)
Blocos comentados de outros provedores com referências a chaves inexistentes	Falha no carregamento, mesmo comentados	Blocos movidos para outro arquivo

4.9 Execução da Indexação

A indexação foi executada em 8 de outubro de 2026, no contêiner do GraphRAG, com o comando `graphrag index -root /data`. A execução durou cerca de 26 minutos, sem erros e sem novas tentativas de chamada, e percorreu os *workflows* do método padrão: leitura das 130 entradas e criação das 130 unidades de texto; extração do grafo com uma rodada adicional (*gleaning*); resumo de 8.270 descrições parciais de entidades e relações; detecção de comunidades; geração dos relatórios de comunidade; e geração de *embeddings* para 400 textos. O paralelismo foi limitado a oito requisições simultâneas por modelo.

4.10 O Grafo Resultante

4.10.1 Visão geral

A Tabela 11 resume o índice gerado. Das 6.885 relações extraídas, 288 (cerca de 4%) foram descartadas pela biblioteca por citarem entidades que não haviam sido extraídas.

Todas as 1.373 entidades pertencem aos catorze tipos da taxonomia, e os 130 artigos do CDC aparecem como entidades no padrão de nomeação definido.

Tabela 11 – Resumo do índice gerado (versão 1).

Item	**Valor**
Documentos / unidades de texto	130 / 130
Entidades	1.373
Relações	6.885
Comunidades (= relatórios)	369
nível 0 / 1 / 2 / 3	17 / 89 / 185 / 78
Artigos do CDC representados como entidades	130
Relações descartadas (entidades inexistentes)	288

A distribuição das entidades por tipo (Tabela 12) reflete o caráter normativo do texto: os tipos mais frequentes são institutos jurídicos, obrigações, dispositivos legais e direitos. As entidades mais conectadas são CONSUMIDOR (grau 597), FORNECEDOR (272), PRODUTO (262) e SERVIÇO (218), o que corresponde aos conceitos centrais da lei.

Tabela 12 – Distribuição das entidades por tipo.

Tipo	**Entidades**	**Tipo**	**Entidades**
`instituto_juridico`	313	`contrato`	57
`obrigacao`	228	`fornecedor`	50
`dispositivo_legal`	177	`produto`	44
`direito`	128	`infracao_penal`	31
`instrumento_processual`	114	`consumidor`	30
`orgao_entidade`	83	`sancao_administrativa`	23
`pratica_abusiva`	80	`servico`	15

4.10.2 Exemplos

Para ilustrar o conteúdo do grafo, considere o direito de arrependimento. A entidade "ART. 49 DO CDC" (tipo `dispositivo_legal`, grau 9) tem a descrição: "Dispositivo vigente do Código de Defesa do Consumidor, situado no capítulo Da Proteção Contratual, seção Disposições Gerais. Assegura ao consumidor o direito de desistir do contrato celebrado fora do estabelecimento comercial em sete dias, contados da assinatura ou do recebimento do produto ou serviço. O parágrafo único determina a devolução imediata e monetariamente atualizada dos valores eventualmente pagos, a qualquer título, durante o prazo de reflexão." A descrição incorpora a localização do artigo na lei, informada pelos metadados da unidade de texto, e registra o prazo na descrição, conforme as regras de nomeação.

A entidade "DIREITO DE ARREPENDIMENTO" (tipo `direito`, grau 14) foi extraída de duas unidades de texto – o art. 49 e o art. 54-F, que trata dos contratos conexos – e sua descrição, resumida pelo modelo, combina as duas fontes. Ela se liga a

CONSUMIDOR por uma relação descrita como “O CONSUMIDOR é titular do DIREITO DE ARREPENDIMENTO previsto no art. 49 [...]”. Esse exemplo mostra o que o grafo acrescenta ao texto: a ligação entre o art. 49 e o art. 54-F, distantes no texto da lei, está explícita no grafo por meio de uma entidade comum.

A Tabela 13 lista as comunidades do nível 0, as mais amplas, com os títulos gerados pelo modelo. Os títulos correspondem a grandes temas do CDC – tutela coletiva, superendividamento, sanções administrativas, tutela penal, vícios do produto, cadastros de consumidores –, o que indica que o particionamento capturou a organização temática da lei. As notas de importância atribuídas pelo modelo são altas e pouco variáveis (entre 8,7 e 9,6), o que limita sua utilidade para ordenar relatórios.

Tabela 13 – Comunidades do nível 0, com os títulos gerados pelo modelo.

Id	Título	Entidades	Nota
5	CDC, associações e tutela coletiva do consumidor	203	9,4
0	Consumidores, Fornecedores e Proteção no CDC	186	9,4
3	CDC, Defesa do Consumidor e Repactuação do Superendividamento	148	9,6
1	Consumidor, garantias contratuais e proteção no crédito	138	9,2
15	Proteção do consumidor: publicidade, segurança e aplicação do CDC	101	9,3
8	Consumidores e fornecedores: responsabilidade, orçamento e serviços públicos no CDC	75	9,4
4	SNDC: coordenação federal, fiscalização e participação no CDC	70	9,5
13	Consumidores, fornecedores e proteção contratual nos arts. 6º e 51 do CDC	69	9,6
9	Proteção judicial do consumidor: arts. 28, 84 e 101 do CDC	63	9,2
2	Sanções administrativas do CDC: autoridades, fornecedores e fundos de proteção	60	9,2
14	Crédito ao consumidor: fornecedores e arts. 54-F e 54-G do CDC	49	9,4
7	Consumidores e fornecedores: vícios nos arts. 18, 19 e 20 do CDC	47	9,4
16	CDC: proteção contratual, responsabilidade penal e atuação do Ministério Público	43	9,2
10	Consumidor inadimplente: cobrança e proteção contratual nos arts. 42, 53 e 71 do CDC	34	9,0
6	CDC: responsabilidade do fornecedor e tutela coletiva por associações	29	9,2
12	Tutela penal do consumidor: arts. 77 a 79 do CDC, juiz e Código Penal	29	8,7
11	Consumidores, cadastros e proteção informacional: arts. 43 e 72 do CDC	27	9,3

4.10.3 Verificações de qualidade

Foram feitas verificações automáticas e por amostragem sobre o grafo: nenhum nome de entidade fora do padrão de maiúsculas; artigos de outras normas também padronizados (por exemplo, “ART. 16 DA LEI Nº 7.347/1985” e “ART. 44 DO CÓDIGO PENAL”); prazos registrados nas descrições, e não como entidades isoladas; e a entidade do art. 54-E registrando que o dispositivo foi vetado e que seria incluído pela Lei nº 14.181/2021. Esses resultados indicam que as definições dos tipos e as regras de nomeação inseridas no *prompt* foram efetivas.

Aluno: Incluir aqui, se desejado, uma análise qualitativa manual de uma amostra de entidades e relações (por exemplo, 30 relações sorteadas, classificadas como corretas, imprecisas ou incorretas).

A Figura 4 mostra o grafo completo na aplicação desenvolvida, colorido por tipo

de entidade.

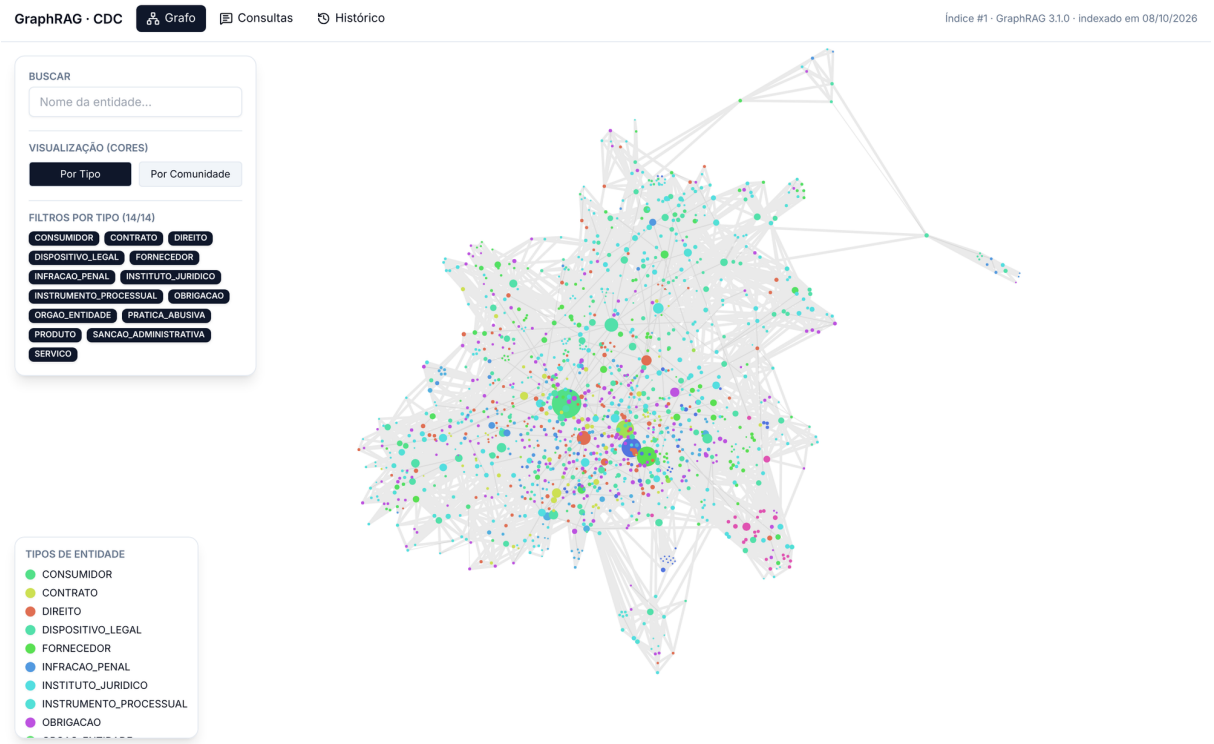


Figura 4 – Grafo de conhecimento do CDC na aplicação web, com as entidades coloridas por tipo.

4.11 Custo da Indexação

A Tabela 14 apresenta o consumo de *tokens* registrado pelo GraphRAG durante a indexação. O modelo principal fez 630 chamadas: 260 de extração (duas por unidade de texto, devido à rodada adicional) e as demais de geração dos 369 relatórios de comunidade. O modelo barato fez 406 chamadas de resumo de descrições.

Tabela 14 – Consumo de *tokens* da indexação.

Modelo	**Chamadas**	***Tokens* de entrada**	***Tokens* de saída**
`gpt-6.1-sol`	630	4.303.693	1.471.243
`gpt-6-luna`	406	188.741	157.098
`text-embedding-3-large`	220	1.032.518	–

O custo estimado pelos preços de tabela dos modelos ficou entre 16 e 24 dólares, enquanto a estimativa feita antes da indexação, a partir do tamanho do corpus, era de 3 a 6 dólares. A diferença veio principalmente dos *tokens* de saída, os mais caros: a extração foi muito mais detalhada do que o previsto, com mais de 6.800 relações para um corpus de 130 artigos, e cada relação é acompanhada de uma descrição. Além disso, o *prompt* de extração, com cerca de 11,7 mil *tokens*, é enviado em todas as chamadas de extração, e os

tokens de raciocínio dos modelos utilizados são cobrados mas não aparecem nas contagens. A lição é que o custo do GraphRAG é dominado pela extração do grafo e pelos *tokens* gerados, e que estimativas baseadas apenas no tamanho do corpus o subestimam.

4.12 Publicação e Reprodutibilidade

Para que o índice possa ser reutilizado sem nova indexação, e para que a avaliação seja feita sobre um índice identificado e imutável, o resultado foi publicado como versão (*release*) do repositório no GitHub, com a marcação `v0.1.0`, seguindo o versionamento semântico. O pacote contém as pastas de saída, cache e registros da indexação e cópias da configuração, dos *prompts* e do corpus, nos caminhos originais, acompanhado de uma soma de verificação SHA-256, de um manifesto (versão do código, modelos, estatísticas do índice, consumo de *tokens* e *hashes* da configuração e dos *prompts*) e de notas da versão com instruções de restauração. Nenhuma chave de API é incluída, o que foi verificado por busca de padrões de chave no pacote. A marcação `v1.0.0` foi reservada para o índice que será usado na avaliação final.

Como o cache das respostas dos modelos faz parte do pacote, quem restaurar a versão pode reexecutar a indexação com a mesma configuração praticamente sem custo, e verificar que o resultado é o mesmo. A versão do GraphRAG foi fixada em 3.1.0 em todos os ambientes, já que tanto o formato dos dados quanto o comportamento das consultas podem mudar entre versões.

5 Aplicação Web para Exploração e Consulta do Grafo

Este capítulo apresenta a aplicação web desenvolvida como parte da prova de conceito. A Seção 5.1 descreve seus objetivos e funcionalidades. A Seção 5.2 apresenta a arquitetura e a Seção 5.3, os contêineres. As seções 5.4 a 5.6 detalham o *backend*, o banco de dados e o *frontend*. A Seção 5.7 reúne os problemas de ambiente enfrentados, e a Seção 5.8, a validação funcional da aplicação.

5.1 Objetivos e Funcionalidades

A primeira versão da aplicação apenas lia os arquivos do índice e exibia o grafo; as consultas eram feitas pela linha de comando, dentro do contêiner do GraphRAG, e mostravam somente a resposta final. Para os objetivos deste trabalho, isso era insuficiente. Comparar as modalidades de busca exige ver **o que cada uma recupera e envia ao modelo**, e não apenas a resposta, e exige registrar as consultas de forma sistemática para a avaliação. A aplicação foi então reestruturada com os seguintes requisitos funcionais:

RF01 – Visualizar o grafo. Exibir o grafo de conhecimento completo, de forma interativa, com coloração por tipo de entidade ou por comunidade, filtros por tipo, busca por nome e painel de detalhes de cada entidade, com sua descrição, suas relações e os relatórios das comunidades a que pertence.

RF02 – Realizar consultas. Permitir fazer perguntas em linguagem natural em qualquer das quatro modalidades (local, global, DRIFT e básica), escolhendo o nível de comunidade e o formato da resposta.

RF03 – Exibir as etapas da consulta. Mostrar, à medida que acontecem, as etapas intermediárias de cada consulta: o contexto recuperado (entidades, relações, relatórios e trechos), o *prompt* enviado ao modelo, as respostas parciais (nas modalidades global e DRIFT), a resposta final e as métricas de chamadas, *tokens* e tempo.

RF04 – Relacionar a consulta ao grafo. Destacar no grafo as entidades e relações correspondentes ao contexto de cada consulta.

RF05 – Registrar o histórico. Armazenar cada consulta, com todas as etapas, em banco de dados, e permitir listá-las, filtrá-las, revê-las e excluí-las.

RF06 – Apoiar a avaliação. Permitir que consultas sejam disparadas por *scripts* de

avaliação pela mesma interface de programação (API) usada pela interface, identificando sua origem, e armazenar as notas atribuídas às respostas.

E os seguintes requisitos não funcionais:

RNF01 – Fidelidade. Uma consulta pela aplicação deve ser equivalente à mesma consulta pela CLI do GraphRAG: mesmo índice, mesma configuração, mesmos *prompts* e mesma versão da biblioteca.

RNF02 – Rastreabilidade. Cada consulta deve registrar qual versão do índice e da configuração a respondeu.

RNF03 – Reprodutibilidade. O sistema deve ser executável em qualquer máquina com Docker, com versões fixadas.

RNF04 – Uso local. Como cada consulta gera custos na conta do provedor de modelos, a aplicação destina-se a uso local e não deve ser exposta publicamente com a chave de API configurada.

5.2 Arquitetura

A Figura 5 apresenta a arquitetura. Há um **índice único**, a pasta `data/`, que contém a configuração, os *prompts*, a chave de API e o resultado da indexação. O contêiner `graphrag` escreve nessa pasta ao indexar; o *backend* lê os mesmos arquivos, com a mesma versão da biblioteca e a mesma configuração, o que garante o requisito de fidelidade (RNF01). O *backend* grava o histórico em um banco SQLite em uma pasta separada (`storage/`), e o *frontend* se comunica apenas com o *backend*. Ambos os componentes que usam modelos – o de indexação e o *backend* – chamam a API do provedor diretamente.

A aplicação suporta três fluxos:

1. **Indexação** (rara): executada pela CLI no contêiner `graphrag`, gera a pasta `data/output`. Após uma nova indexação, o *backend* recarrega o índice por uma rota da API ou ao ser reiniciado.
2. **Consulta** (interativa): o navegador envia a pergunta ao *backend*, que monta o motor de busca do GraphRAG, executa a consulta e envia cada etapa ao navegador por SSE, gravando-a ao mesmo tempo no banco.
3. **Avaliação** (em lote): um *script* chama a mesma API para um conjunto de perguntas, modalidades e repetições, identificando as consultas como de avaliação; os resultados são exportados do banco para análise.

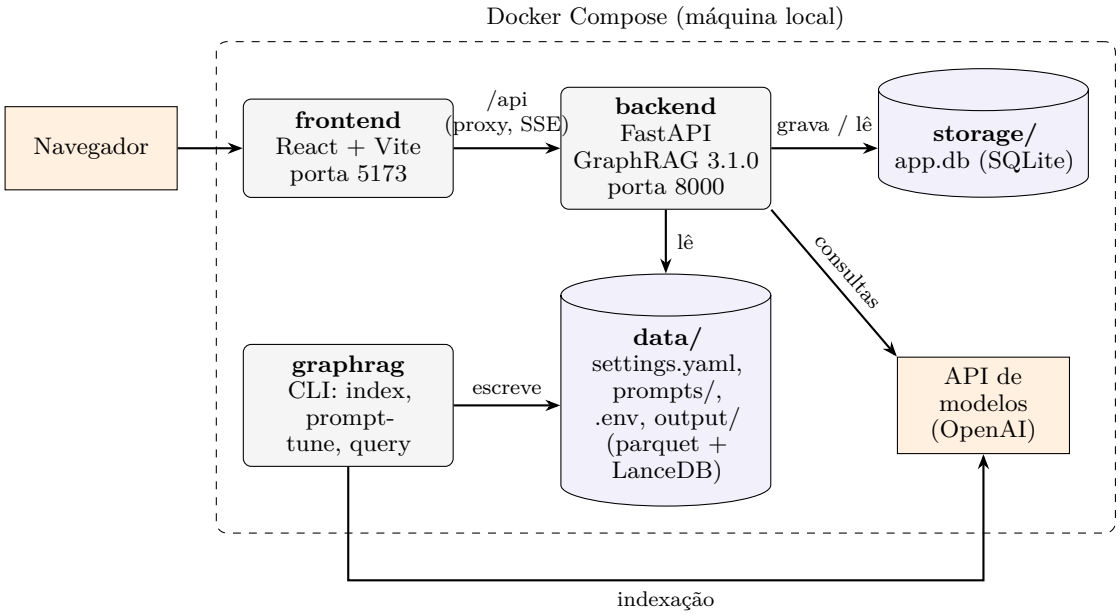


Figura 5 – Arquitetura da aplicação.

5.3 Contêineres e Execução

O sistema é composto pelos serviços da Tabela 15, definidos em um único arquivo `docker-compose.yml`. As pastas do projeto são montadas nos contêineres como volumes, de modo que o índice, a configuração e o banco persistem fora dos contêineres e são compartilhados entre eles. A versão do GraphRAG é fixada (3.1.0) tanto na imagem de indexação quanto na do *backend*, já que o *backend* utiliza funções internas da biblioteca, que podem mudar entre versões.

Tabela 15 – Serviços do Docker Compose.

Serviço	**Imagem**	**Volumes**	**Função**
`graphrag`	Python 3.11 com `graphrag==3.1.0`	`data/`, `scripts/`, `corpus/`	Indexação, ajuste de *prompts*, consultas pela CLI e *scripts* auxiliares. Não é um servidor: é usado sob demanda.
`backend`	Python 3.11 com FastAPI e `graphrag==3.1.0`	`data/`, `storage/`	API: grafo, consultas com etapas, histórico e versão do índice (porta 8000).
`frontend`	Node.js 20 com Vite	código do *frontend*	Interface web (porta 5173); repassa `/api` ao *backend*.
`ollama`	Ollama	modelos locais	Servidor de modelos locais. Só é iniciado com o perfil `local`.

A chave de API fica no arquivo `data/.env`, lido automaticamente pelo GraphRAG e excluído do controle de versão; um modelo do arquivo, sem a chave, acompanha o repositório. Um *script* de configuração para Windows automatiza a preparação do ambiente: cria o `.env` a partir do modelo, constrói as imagens, inicia os serviços e, opcionalmente, executa a indexação ou usa a configuração local.

5.4 *Backend*

5.4.1 Organização

O *backend* é uma aplicação FastAPI organizada em configuração, modelos de dados, serviços e rotas (Tabela 16). Ao iniciar, cria as tabelas do banco, se necessário, carrega o índice e registra a versão do índice em uso.

Tabela 16 – Módulos do *backend*.

Módulo	**Responsabilidade**
`config.py`	Caminhos (raiz do GraphRAG e pasta do banco) e parâmetros padrão das consultas, iguais aos da CLI (nível de comunidade 2 e resposta em "*Multiple Paragraphs*").
`db.py`, `models.py`	Conexão com o SQLite e definição das tabelas com o SQLAlchemy.
`services/query_service.py`	Carrega a configuração, as tabelas do índice e os *prompts*; monta os motores de busca; rastreia e emite as etapas de cada consulta.
`services/run_service.py`	Cria o registro da consulta, executa-a em segundo plano, grava as etapas e as repassa ao navegador.
`services/index_service.py`	Calcula a impressão digital do índice e registra suas versões.
`services/graph_loader.py`	Lê o índice e monta o grafo para visualização.
`routers/`	Rotas da API: grafo, consultas, histórico e índice.

5.4.2 Carregamento do índice e montagem dos motores

Para garantir a equivalência com a CLI, o *backend* reproduz o que a própria biblioteca faz ao executar `graphrag query`: carrega a configuração com a função da biblioteca, a partir da mesma pasta raiz; lê as tabelas do índice com o mesmo leitor de dados; e monta os motores de busca com as mesmas funções de fábrica (*factory*) usadas pela API da biblioteca, com os mesmos parâmetros. Os *prompts* de consulta são lidos a partir da pasta raiz, e não do diretório corrente. Se um arquivo de *prompt* configurado não existir, o *backend* falha com uma mensagem clara, em vez de usar silenciosamente o *prompt* padrão, como faria a biblioteca (defeito 4 da Tabela 6).

Os motores de busca do GraphRAG oferecem dois métodos: `stream_search`, usado pela CLI, que devolve apenas o texto da resposta à medida que é gerado; e `search`, que devolve o resultado completo, incluindo o contexto, o número de chamadas e os *tokens* por etapa. O *backend* usa `search`, porque precisa dessas informações para exibi-las e registrá-las. A resposta continua sendo exibida à medida que é gerada, porque `search` também notifica cada fragmento de texto gerado.

5.4.3 Rastreamento das etapas

O GraphRAG permite registrar objetos de *callback* nos motores de busca, que são notificados em pontos do processamento: quando o contexto é montado, no início e no fim das etapas map e reduce e a cada fragmento de resposta gerado. O *backend* implementa um *callback* que converte cada notificação em um evento, conforme a Tabela 17.

Tabela 17 – Eventos emitidos durante uma consulta.

Evento	**Modalidades**	**Conteúdo**
`run`	todas	Identificador da consulta, parâmetros e versão do índice.
`context`	todas	Tabelas recuperadas (entidades, relações, relatórios, trechos) e o texto exato inserido no *prompt*.
`prompt`	local, básica, global	Mensagem de sistema final e pergunta enviadas ao modelo (na global, o *prompt* da síntese).
`map_start`, `map_end`	global	Lotes de relatórios e pontos-chave, com notas, de cada lote.
`reduce_start`, `reduce_end`	global, DRIFT	Entrada e saída da síntese; na DRIFT, a árvore de perguntas de acompanhamento.
`token`	todas	Fragmentos da resposta conforme gerada (não gravado no banco).
`result`	todas	Resposta final, chamadas ao modelo, *tokens* de entrada e saída por etapa e tempo.
`error`, `done`	todas	Erro ou fim da consulta.

Os *callbacks* da biblioteca, porém, não bastam para exibir o contexto antes da resposta: nas buscas local e básica, o GraphRAG só notifica o contexto depois de gerar a resposta (defeito 9 da Tabela 6). Para contornar isso sem modificar a biblioteca, o *backend* substitui, na instância do motor criada para cada consulta, o método que monta o contexto por uma versão que chama o método original e, em seguida, emite o contexto e o *prompt* final. O *prompt* é reconstruído com o mesmo modelo de texto e os mesmos argumentos que o GraphRAG usa, de modo que o usuário vê exatamente o que é enviado ao modelo, antes mesmo de a resposta começar.

5.4.4 Execução em segundo plano e transmissão

Uma consulta começa com uma requisição `POST /api/query`. O *backend* cria o registro da consulta no banco, com estado "em andamento", e inicia a execução em uma tarefa assíncrona separada da conexão. Cada evento emitido pela tarefa é gravado no banco (exceto os fragmentos de resposta, para não sobrecarregá-lo) e colocado em uma fila, da qual a rota lê e envia os eventos ao navegador no formato SSE. Ao final, o registro recebe a resposta, as métricas e o estado final. Como a gravação acontece na tarefa, e não na conexão, uma consulta iniciada continua e fica registrada mesmo que o navegador seja fechado – o que é importante para consultas que levam minutos e custam dinheiro. Para os *scripts* de avaliação, a mesma rota aceita um parâmetro que desativa o *streaming* e

devolve o registro completo ao final.

5.4.5 Correção da busca básica

O primeiro teste da busca básica pela aplicação devolveu uma resposta vazia, sem erro. A investigação no código da biblioteca mostrou a causa (defeito 1 da Tabela 6): o método `search()` da busca básica chama a geração do modelo em modo *streaming* sem o `await` necessário e percorre o resultado como um iterador assíncrono. O erro resultante é capturado internamente, e a busca devolve uma resposta vazia. A CLI não é afetada porque usa `stream_search()`, que tem o `await` correto. O defeito também está presente na versão 3.2.0.

Como o *backend* não deve modificar a biblioteca, a correção foi feita por um invólucro (Listagem 5.1) em torno do modelo entregue ao motor da busca básica: quando o motor pede uma geração em modo *streaming*, o invólucro devolve um iterador assíncrono que primeiro aguarda a chamada e depois repassa os fragmentos. Todo o resto – *prompt*, trechos recuperados, modelo e parâmetros – permanece idêntico ao da CLI. Esse achado tem consequência direta para trabalhos que usem a biblioteca programaticamente: sem a correção, o *baseline* vetorial "responderia" sempre vazio, o que invalidaria qualquer comparação com as modalidades baseadas em grafo.

Listagem 5.1 – Invólucro que corrige a busca básica do GraphRAG 3.1.0.

```
class _AwaitedStreamModel:
    """Faz a chamada em streaming sem await devolver um iterador assíncrono."""
    def __init__(self, model):
        self._model = model

    def __getattr__(self, name):            # demais atributos: os do modelo original
        return getattr(self._model, name)

    def completion_async(self, **kwargs):
        if not kwargs.get("stream"):
            return self._model.completion_async(**kwargs)
        async def _stream():
            response = await self._model.completion_async(**kwargs)
            async for chunk in response:
                yield chunk
        return _stream()
```

5.4.6 Versionamento do índice

Para atender ao requisito de rastreabilidade (RNF02), o *backend* calcula, ao carregar o índice, uma impressão digital: um *hash* SHA-256 que combina os *hashes* das tabelas principais do índice, do `settings.yaml` e de todos os *prompts*. Se qualquer um desses arquivos mudar – uma reindexação, uma alteração de configuração das buscas ou de um

prompt –, a impressão digital muda e uma nova versão do índice é registrada no banco, com a data da indexação, os modelos e a versão da biblioteca. Cada consulta guarda a versão que a respondeu. Assim, resultados obtidos com configurações diferentes nunca se misturam na avaliação.

5.4.7 Rotas da API

A Tabela 18 lista as rotas da API. A documentação interativa é gerada automaticamente pelo FastAPI.

Tabela 18 – Rotas da API do *backend*.

Método	**Rota**	**Descrição**
GET	`/api/graph`, `/api/communities`, `/api/meta`	Grafo para visualização, comunidades e metadados (tipos, níveis). Cada nó traz as unidades de texto de onde foi extraído.
POST	`/api/query`	Executa uma consulta e transmite as etapas por SSE (ou devolve o registro completo, sem *streaming*).
GET	`/api/runs`	Histórico, com filtros por modalidade, origem, estado, rodada de avaliação e texto.
GET, DELETE	`/api/runs/{id}`	Consulta com todas as etapas; exclusão.
GET	`/api/index`	Versão do índice em uso e estado do serviço de consultas.
POST	`/api/index/reload`	Recarrega o índice após uma reindexação.
GET	`/api/health`	Verificação de funcionamento.

5.5 Banco de Dados

O banco SQLite tem seis tabelas, apresentadas na Figura 6. O SQLite foi escolhido por não exigir servidor, ser armazenado em um único arquivo – fácil de copiar, versionar como resultado e inspecionar – e ser suficiente para o volume de dados de uma prova de conceito. O banco opera no modo WAL (*write-ahead logging*), que permite leituras simultâneas às gravações feitas pelas consultas em andamento.

- `index_versions`: versões do índice, identificadas pela impressão digital (Seção 5.4.6);
- `query_runs`: uma linha por consulta, com pergunta, modalidade, parâmetros, origem (interface, avaliação ou API), versão do índice, estado, resposta e métricas, inclusive o detalhamento de chamadas e *tokens* por etapa;
- `query_steps`: as etapas de cada consulta, na ordem em que ocorreram, com o conteúdo de cada evento em JSON;

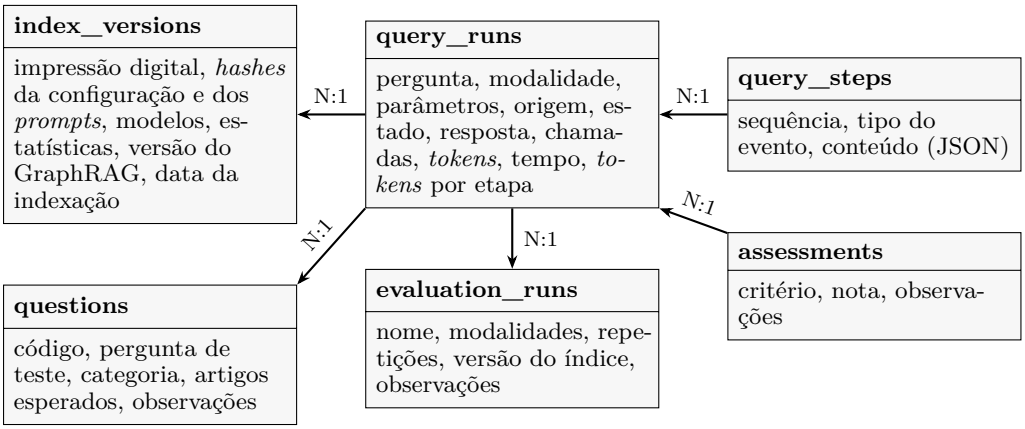


Figura 6 – Tabelas do banco de dados da aplicação.

- `questions`, `evaluation_runs` e `assessments`: perguntas de teste, rodadas de avaliação (modalidades e repetições) e notas atribuídas às respostas, que serão utilizadas na avaliação (Capítulo 6).

5.6 *Frontend*

O *frontend* é uma aplicação de página única com três abas – Grafo, Consultas e Histórico – e um cabeçalho que mostra a versão do índice em uso. As três telas permanecem carregadas e são apenas ocultadas ao trocar de aba; assim, é possível, por exemplo, iniciar uma consulta demorada e explorar o grafo enquanto ela é executada. O grafo é carregado uma única vez e compartilhado entre as telas.

5.6.1 Comunicação com o *backend*

O servidor do Vite atua como *proxy*: as requisições do navegador para `/api` são repassadas ao contêiner do *backend*. Assim, *frontend* e API são vistos pelo navegador como a mesma origem, o que dispensa configuração de compartilhamento entre origens (CORS) e permite o fluxo de eventos. Os eventos SSE são lidos a partir da resposta da requisição de consulta e despachados para o componente que exibe a consulta.

5.6.2 Aba Grafo

A aba Grafo (Figura 4, no Capítulo 4) exibe as 1.373 entidades e 6.885 relações com leiaute por simulação de forças. O tamanho de cada nó é proporcional ao seu grau. O painel lateral permite buscar entidades pelo nome, alternar a coloração entre tipo de entidade e comunidade (com escolha do nível da hierarquia, Figura 7) e filtrar os tipos exibidos. Ao clicar em uma entidade, um painel mostra seu tipo, sua descrição, suas relações e os relatórios das comunidades a que pertence.

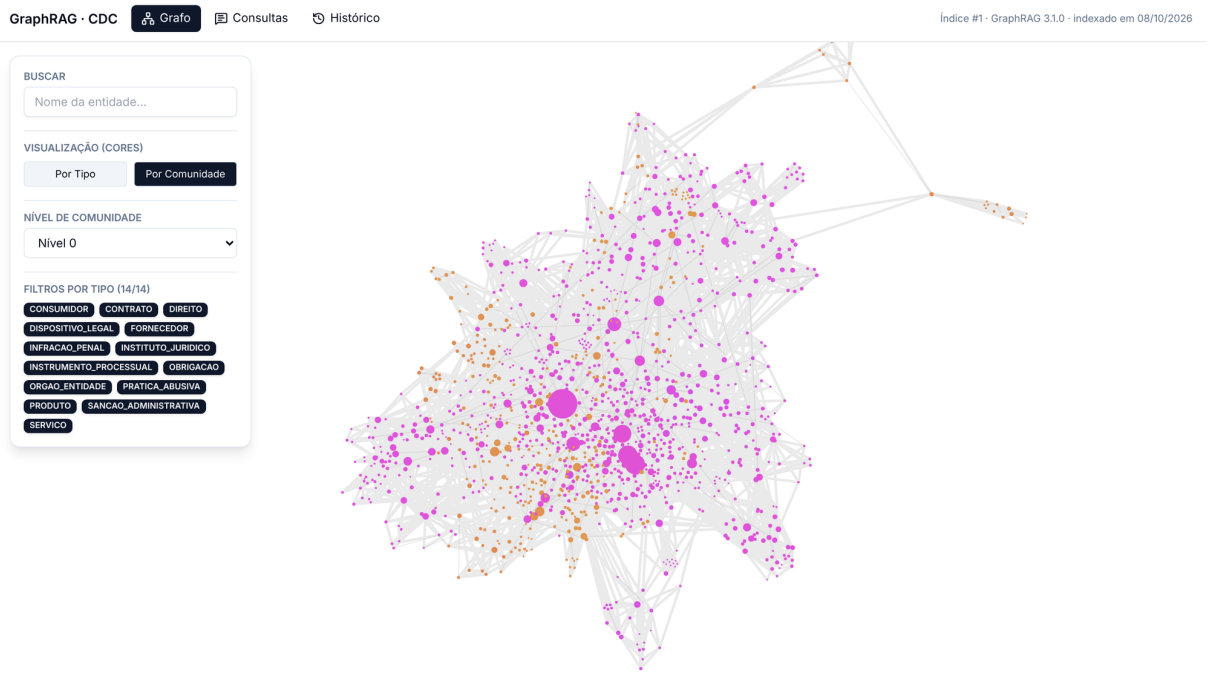


Figura 7 – Aba Grafo com as entidades coloridas por comunidade.

5.6.3 Aba Consultas

A aba Consultas (Figura 8) tem um formulário com a pergunta, a modalidade (com uma descrição de cada uma), o nível de comunidade, com um aviso de custo quando a busca global é escolhida, e o formato da resposta, além de perguntas de exemplo. Ao enviar, a consulta é exibida como uma linha do tempo de etapas recolhíveis, que se completam à medida que os eventos chegam (Figura 9). As etapas variam conforme a modalidade:

- **Recuperação de contexto** (local, básica e DRIFT; na global, **relatórios considerados**): o contexto em abas – entidades, relações, relatórios, trechos e o texto exato enviado ao modelo (Figura 10);
- **Map** (global): os pontos-chave de cada lote, ordenados pela nota e com o lote de origem;
- **Subgrafo usado como contexto**: o destaque no grafo, descrito a seguir;
- **Raciocínio DRIFT** (DRIFT): a árvore de perguntas de acompanhamento, com a resposta parcial de cada uma;
- ***Prompt* enviado ao modelo** (local, básica e global): a mensagem de sistema completa e a pergunta, com botão para copiar;
- **Resposta**: exibida à medida que é gerada, com formatação;
- **Métricas**: chamadas ao modelo, *tokens* de entrada e de saída e tempo, no total e por etapa.

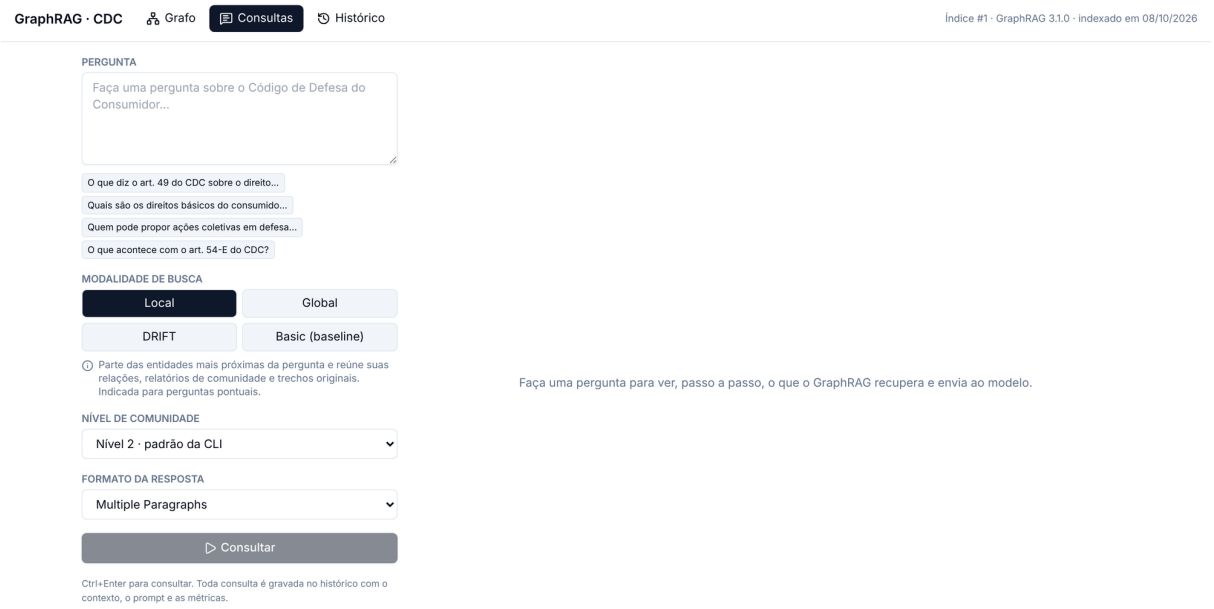


Figura 8 – Aba Consultas: formulário com modalidade, nível de comunidade, formato da resposta e perguntas de exemplo.



Figura 9 – Linha do tempo de uma consulta real na busca local: etapas recolhíveis, resposta com citações e métricas.

5.6.4 Aba Histórico

A aba Histórico (Figura 11) lista as consultas registradas, com data, modalidade, pergunta, estado, número de chamadas, *tokens* e tempo, e permite filtrar por modalidade, origem e texto. Ao abrir uma consulta, a mesma linha do tempo da aba Consultas é reconstruída a partir das etapas gravadas no banco.

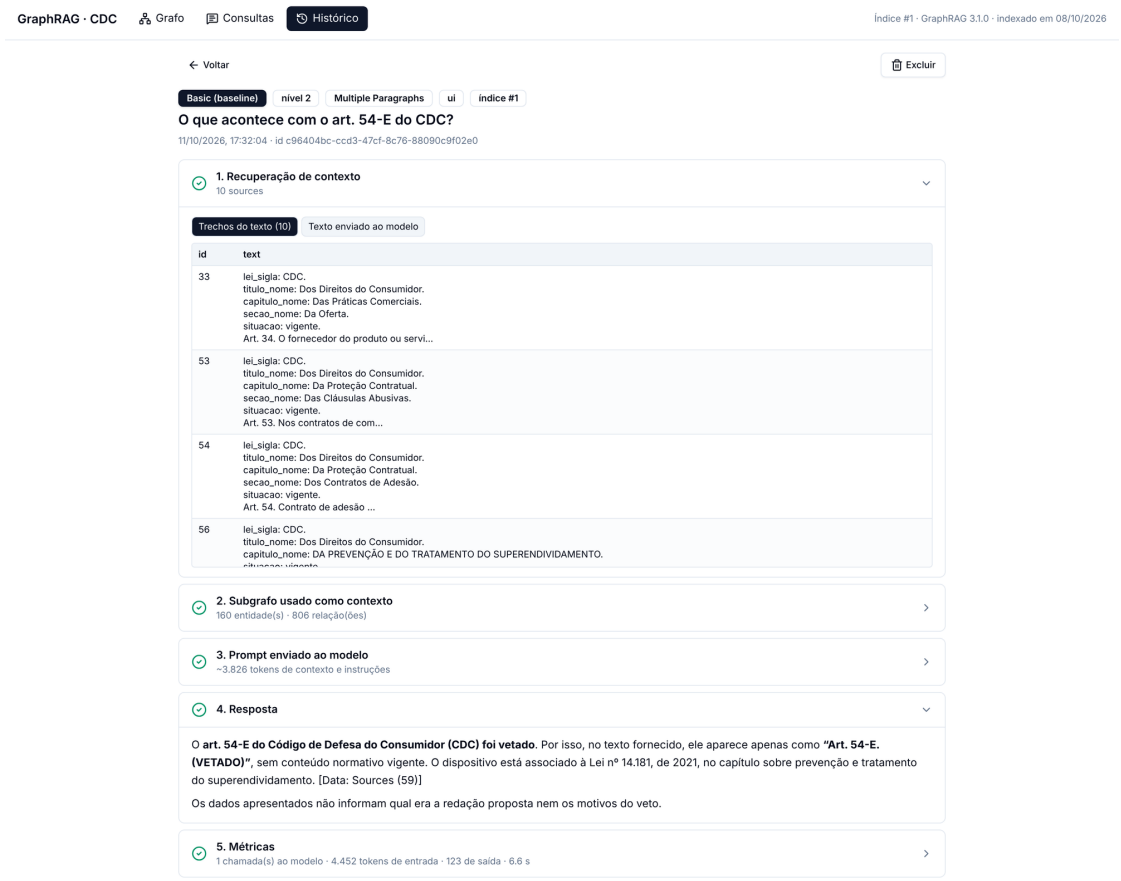


Figura 10 – Contexto recuperado por uma consulta real na busca básica: os trechos (artigos) recuperados por similaridade, em ordem.

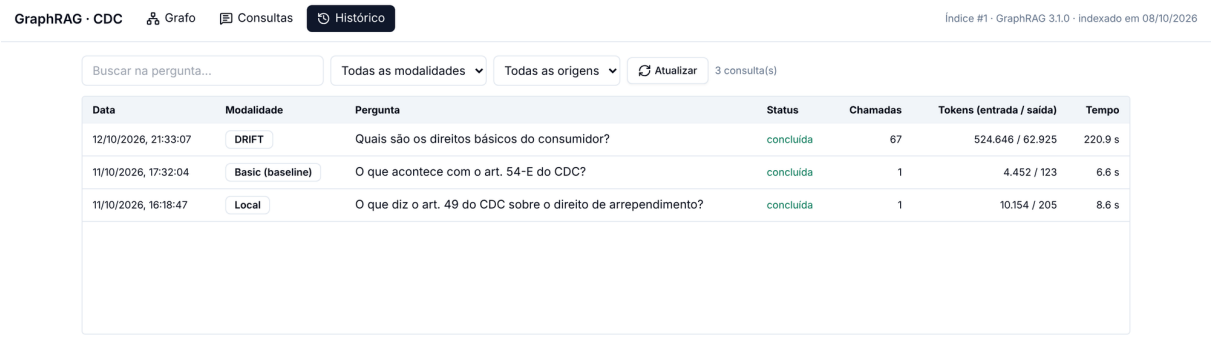


Figura 11 – Aba Histórico com consultas reais nas buscas DRIFT, básica e local.

5.6.5 Destaque do contexto no grafo

Para apoiar a análise qualitativa – entender por que uma resposta saiu mais ou menos completa –, cada consulta mostra qual parte do grafo corresponde ao seu contexto, em duas formas: um subgrafo interativo na própria linha do tempo e, pelo botão "Ver no grafo completo", o destaque dessas entidades e relações no grafo inteiro, com as demais esmaecidas, as relações destacadas em vermelho e o enquadramento automático nos nós destacados (Figura 12).

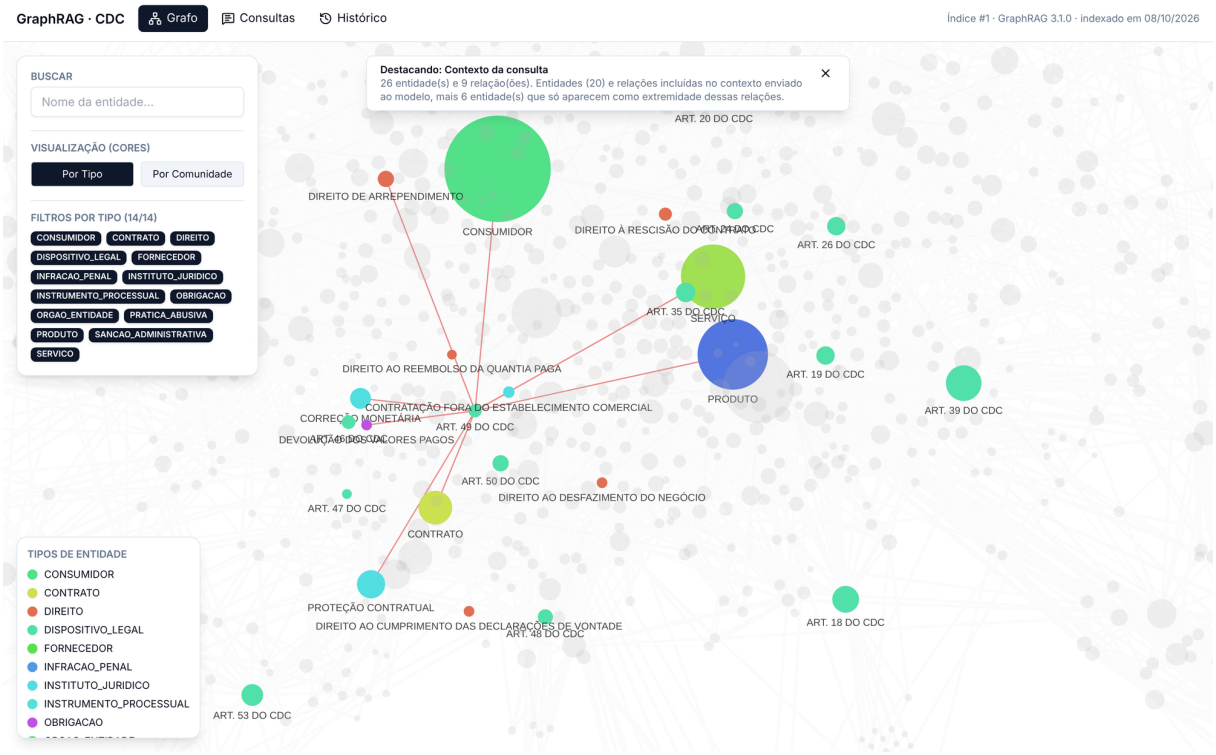


Figura 12 – Destaque, no grafo completo, das entidades e relações usadas como contexto em uma consulta real na busca local sobre o art. 49 do CDC.

O cálculo depende da modalidade, porque cada uma recupera um tipo diferente de informação:

- **Local e DRIFT**: as entidades da tabela de entidades do contexto, localizadas no grafo pelo nome, e as relações da tabela de relações, incluindo as entidades que só aparecem como extremidade dessas relações. Na DRIFT, usa-se o contexto da primeira busca local, pois a biblioteca não expõe o contexto de cada pergunta de acompanhamento.
- **Básica**: a busca básica não usa o grafo; o destaque mostra as entidades **extraídas dos trechos recuperados**, isto é, o que o grafo sabe sobre os mesmos trechos que o modelo recebeu como texto. Para isso, o *backend* passou a informar, para cada nó do grafo, os identificadores curtos das unidades de texto de onde a entidade foi extraída – os mesmos que aparecem no contexto e nas citações (por exemplo, a entidade "ART. 54-E DO CDC" aponta para a unidade 59, a mesma citada pela busca básica na pergunta sobre esse artigo).
- **Global**: as entidades das comunidades cujos relatórios foram citados nos pontos-chave relevantes da etapa map e na resposta final.

O contraste entre as modalidades é informativo. Na busca local sobre o art. 49, o contexto é um conjunto pequeno de 26 entidades e 9 relações em torno do artigo (Figura 12);

na básica, os dez trechos recuperados por similaridade "tocam" uma região de centenas de entidades, mas o modelo recebe apenas o texto; na global, o contexto é definido por comunidades inteiras.

5.7 Problemas de Ambiente

Além dos defeitos da biblioteca, o desenvolvimento enfrentou problemas de ambiente que vale registrar, por serem comuns em projetos semelhantes:

- **Página em branco.** Após reconstruir a imagem do *frontend*, a aplicação passou a exibir uma página em branco. O arquivo de dependências pedia a versão 4 do Tailwind CSS, mas o código havia sido escrito para a versão 3, cuja forma de configuração é diferente; o processamento do CSS falhava e derrubava a aplicação. A versão foi fixada na série 3.
- **Dependências da máquina dentro da imagem.** A correção acima não surtia efeito porque, sem um arquivo `.dockerignore`, a cópia do código para a imagem incluía a pasta de dependências da máquina local, sobrescrevendo a instalação limpa feita na imagem. Foram criados arquivos `.dockerignore` para o *frontend* e o *backend*.
- **Recursão infinita na árvore da DRIFT.** O estado da busca DRIFT pode conter arestas repetidas e laços (uma pergunta ligada a si mesma); sem tratamento, a renderização da árvore entrava em recursão infinita e travava a página. A árvore passou a ignorar laços e arestas repetidas.

5.8 Validação Funcional

A aplicação foi validada em duas etapas. Na primeira, para não gerar custos durante o desenvolvimento, utilizou-se o índice real com os modelos simulados (*mock*) oferecidos pela própria biblioteca, aos quais foi acrescentado suporte a *streaming* apenas no ambiente de teste. Esse ambiente exercitou as quatro modalidades, o fluxo de eventos, a gravação no banco, o histórico e a interface, com testes automatizados em navegador e capturas de tela. Foi nessa etapa que se identificaram a falha da busca básica e a recursão da árvore da DRIFT.

Na segunda etapa, foram feitas consultas reais, com custo, em cada modalidade de busca já habilitada, resumidas na Tabela 19. A busca local sobre o art. 49 respondeu corretamente (prazo de sete dias, contratação fora do estabelecimento comercial, devolução imediata e atualizada dos valores), citando a unidade de texto do artigo. A busca básica sobre o art. 54-E, que validou a correção da Seção 5.4.5, respondeu corretamente que o artigo foi vetado – informação disponível apenas porque o texto vetado foi mantido no

corpus. Nessa consulta, os primeiros trechos recuperados por similaridade foram os dos arts. 34 e 52, e não o do art. 54-E, um exemplo da limitação de que similaridade não é relevância (Seção 2.3).

Tabela 19 – Consultas reais de validação.

Busca	**Pergunta**	**Chamadas**	**Entrada**	**Saída**	**Tempo**
Local	O que diz o art. 49 do CDC sobre o direito de arrependimento?	1	10.154	205	8,6 s
Básica	O que acontece com o art. 54-E do CDC?	1	4.452	123	6,6 s
DRIFT	Quais são os direitos básicos do consumidor?	67	524.646	62.925	220,9 s

A consulta DRIFT, feita com a configuração padrão, respondeu corretamente com base no art. 6º, mas sua análise detalhada revelou dois pontos que motivaram uma revisão da configuração antes da avaliação. O primeiro é o custo: 67 chamadas e mais de meio milhão de *tokens* de entrada, cerca de cinquenta vezes uma busca local. O segundo é que, nas 60 buscas locais executadas pela DRIFT, o contexto não continha nenhuma entidade, relação ou relatório, apenas trechos de texto, em razão das proporções padrão de contexto (defeito 7 da Tabela 6). A árvore de perguntas tinha 502 nós, dos quais apenas 61 foram respondidos, o que corresponde ao funcionamento esperado do algoritmo. As decisões de configuração decorrentes dessa análise são tratadas no Capítulo 6.

Aluno: A busca global ainda não foi executada com o modelo real; incluir aqui quando for.

6 Avaliação Comparativa das Modalidades de Busca

Aluno: Capítulo a ser escrito após a execução da avaliação. A estrutura abaixo é uma proposta, com o que já foi decidido ou preparado.

Este capítulo apresentará a avaliação comparativa entre as modalidades de busca do GraphRAG (local, global e DRIFT) e o *baseline* vetorial (busca básica).

6.1 Objetivo e Questões de Avaliação

[Definir as questões: as modalidades baseadas em grafo produzem respostas mais completas e corretas que o *baseline*? Para que tipos de pergunta? A que custo? Quão consistentes são entre repetições?]

6.2 Planejamento

O planejamento já definido durante o desenvolvimento inclui:

- **Índice congelado**: todas as modalidades consultam o mesmo índice, identificado pela impressão digital registrada pelo *backend* e publicado como versão `v1.0.0` do repositório;
- **Mesmos modelos e mesma temperatura** em todas as modalidades (Seção 4.5);
- **Repetições** de cada pergunta em cada modalidade, para medir a consistência, já que a temperatura 1, o embaralhamento da busca global e os sorteios da DRIFT fazem as respostas variarem;
- **Execução pela mesma API** usada pela interface, com as consultas identificadas como de avaliação e todas as etapas registradas no banco;
- **Registro de custo**: chamadas e *tokens* por consulta e por etapa, com a ressalva de que os *tokens* de raciocínio não são contabilizados pela biblioteca.

[Conjunto de perguntas: quantidade, categorias (pontuais sobre um artigo, relacionais entre artigos, globais sobre o código, sobre dispositivos vetados), artigos esperados em cada resposta.]

[Critérios de avaliação e escala: correção jurídica, completude, fundamentação nas fontes citadas, ausência de alucinação; avaliação manual e/ou por LLM.]

6.3 Configuração das Buscas

[Registrar as decisões: nível de comunidade da busca global (custo medido por nível: 0 = 7 chamadas e 71 mil *tokens*; 1 = 30 e 336 mil; 2 = 68 e 775 mil; 3 = 82 e 944 mil); proporções de contexto das buscas locais da DRIFT (defeito 7 da Tabela 6); número de perguntas por rodada e de rodadas da DRIFT. A análise das opções está em `docs/notes/configuracao-buscas.md`.]

6.4 Resultados

[Resultados por modalidade e por categoria de pergunta; custo; consistência; exemplos qualitativos com o destaque do contexto no grafo.]

6.5 Discussão e Ameaças à Validade

[Discussão; ameaças à validade: tamanho do conjunto de perguntas, subjetividade da avaliação, um único domínio e um único provedor de modelos, variação entre execuções, versão específica da biblioteca.]

7 Conclusão

Aluno: Capítulo a ser concluído após a avaliação. Os pontos abaixo são um rascunho com o que já pode ser afirmado sobre o desenvolvimento.

7.1 Considerações Finais

Este trabalho aplicou o *framework* GraphRAG ao Código de Defesa do Consumidor, construindo uma prova de conceito que compreende a estruturação do texto da lei, a indexação em grafo de conhecimento e uma aplicação web para exploração do grafo e realização de consultas nas quatro modalidades da ferramenta, com registro detalhado de cada etapa.

Algumas lições do desenvolvimento já podem ser registradas:

- **A preparação do corpus importa.** Estruturar a lei em um registro por artigo, com a hierarquia e a situação normativa inseridas em cada trecho, fez com que cada unidade de texto correspondesse a um artigo completo e permitiu que o grafo registrasse corretamente localização, vetos e revogações.
- **A qualidade do grafo depende da padronização.** Como o GraphRAG funde entidades pelo nome e pelo tipo, as definições dos tipos e as regras de nomeação inseridas no *prompt* de extração foram decisivas para evitar nós duplicados.
- **O custo é dominado pela extração e pelas modalidades com muitas chamadas.** A indexação custou várias vezes a estimativa feita a partir do tamanho do corpus, e as buscas global e DRIFT custam dezenas de vezes uma busca local, conforme a configuração.
- **Usar a biblioteca exige ler o código.** Defeitos e comportamentos não documentados – como a busca básica que devolve resposta vazia quando usada programaticamente, as buscas locais da DRIFT que não usam o grafo com a configuração padrão e o ajuste de *prompts* que gera todos os exemplos a partir do mesmo trecho – afetam diretamente a comparação entre as modalidades e só foram identificados pela inspeção do código e dos dados intermediários. A aplicação desenvolvida, ao expor cada etapa das consultas, foi o que tornou possível identificá-los.

[Relacionar os resultados da avaliação aos objetivos da Seção 1.2; limitações; dificuldades; experiência adquirida.]

7.2 Trabalhos Futuros

[Rascunho de possibilidades:]

- ampliar o corpus com outras leis relacionadas (por exemplo, a Lei da Ação Civil Pública e regulamentações do CDC) e com jurisprudência, aproveitando o *parser* genérico e as remissões entre normas já representadas no grafo;
- explorar a extração de alegações (*claims*) e a seleção dinâmica de comunidades;
- comparar provedores e modelos de linguagem, inclusive modelos locais, mantendo o restante da configuração;
- reportar à equipe do GraphRAG os defeitos identificados, com as correções propostas;
- avaliar automaticamente as respostas com um LLM como juiz, comparando com a avaliação manual.

Referências

Apache Software Foundation. *Apache Parquet*. 2026. <https://parquet.apache.org/>. Acesso em: 13 out. 2026. Citado na página 24.

ASTURIANO, V. *react-force-graph: React component for 2D, 3D, VR and AR force directed graphs*. 2026. <https://github.com/vasturiano/react-force-graph>. Acesso em: 13 out. 2026. Citado na página 26.

BAYER, M. *SQLAlchemy: The Database Toolkit for Python*. 2026. <https://www.sqlalchemy.org/>. Acesso em: 13 out. 2026. Citado na página 25.

BERRIAI. *LiteLLM: call 100+ LLM APIs in OpenAI format*. 2026. <https://github.com/BerriAI/litellm>. Acesso em: 13 out. 2026. Citado 2 vezes nas páginas 24 e 27.

BLONDEL, V. D. et al. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, v. 2008, n. 10, p. P10008, 2008. Disponível em: <https://doi.org/10.1088/1742-5468/2008/10/P10008>. Citado na página 21.

BRASIL. *Lei nº 8.078, de 11 de setembro de 1990. Dispõe sobre a proteção do consumidor e dá outras providências*. 1990. Diário Oficial da União. Brasília, DF, 12 set. 1990. Disponível em: <https://www.planalto.gov.br/ccivil_03/leis/l8078.htm>. Citado 3 vezes nas páginas 12, 23 e 44.

BRASIL. *Lei Complementar nº 95, de 26 de fevereiro de 1998. Dispõe sobre a elaboração, a redação, a alteração e a consolidação das leis*. 1998. Diário Oficial da União. Brasília, DF, 27 fev. 1998. Disponível em: <https://www.planalto.gov.br/ccivil_03/leis/lcp/lcp95.htm>. Citado 2 vezes nas páginas 22 e 44.

BRASIL. *Lei nº 14.181, de 1º de julho de 2021. Altera a Lei nº 8.078, de 11 de setembro de 1990 (Código de Defesa do Consumidor), para aperfeiçoar a disciplina do crédito ao consumidor e dispor sobre a prevenção e o tratamento do superendividamento*. 2021. Diário Oficial da União. Brasília, DF, 2 jul. 2021. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2019-2022/2021/lei/l14181.htm>. Citado 3 vezes nas páginas 13, 23 e 45.

BROWN, T. B. et al. Language models are few-shot learners. *Advances in neural information processing systems*, v. 33, p. 1877–1901, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>. Citado 2 vezes nas páginas 12 e 17.

CHALKIDIS, I. et al. LEGAL-BERT: The muppets straight out of law school. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. [s.n.], 2020. p. 2898–2904. Disponível em: <https://aclanthology.org/2020.findings-emnlp.261>. Citado na página 24.

CUI, J. et al. Chatlaw: Open-source legal large language model with integrated external knowledge bases. *arXiv preprint arXiv:2306.16092*, 2023. Disponível em: <https://arxiv.org/abs/2306.16092>. Citado 2 vezes nas páginas 13 e 24.

Docker Inc. *Docker documentation*. 2026. <https://docs.docker.com/>. Acesso em: 13 out. 2026. Citado na página 26.

EDGE, D. et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. Disponível em: <https://arxiv.org/abs/2404.16130>. Citado 4 vezes nas páginas 12, 20, 22 e 33.

FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado) — University of California, Irvine, 2000. Citado na página 25.

FORTUNATO, S. Community detection in graphs. *Physics Reports*, v. 486, n. 3–5, p. 75–174, 2010. Disponível em: <https://doi.org/10.1016/j.physrep.2009.11.002>. Citado na página 21.

GAO, L. et al. Precise zero-shot dense retrieval without relevance labels. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. [s.n.], 2023. p. 1762–1777. Disponível em: <https://aclanthology.org/2023.acl-long.99>. Citado 2 vezes nas páginas 20 e 33.

GAO, Y. et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023. Disponível em: <https://arxiv.org/abs/2312.10997>. Citado 2 vezes nas páginas 12 e 19.

HOGAN, A. et al. Knowledge graphs. *ACM Computing Surveys*, v. 54, n. 4, p. 1–37, 2021. Disponível em: <https://doi.org/10.1145/3447772>. Citado na página 21.

HOLTZMAN, A. et al. The curious case of neural text degeneration. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2020. Disponível em: <https://arxiv.org/abs/1904.09751>. Citado na página 18.

JI, Z. et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, v. 55, n. 12, p. 1–38, 2023. Disponível em: <https://doi.org/10.1145/3571730>. Citado 2 vezes nas páginas 12 e 18.

JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, IEEE, v. 7, n. 3, p. 535–547, 2019. Disponível em: <https://www.computer.org/csdl/journal/bd/2021/03/08733051/1aFvgKKpjoc>. Citado na página 19.

KARPUKHIN, V. et al. Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [s.n.], 2020. p. 6769–6781. Disponível em: <https://aclanthology.org/2020.emnlp-main.550>. Citado na página 19.

LANCEDB. *LanceDB: Developer-friendly, serverless vector database*. 2024. <https://lancedb.com/>. Acesso em: 16 jul. 2026. Citado 2 vezes nas páginas 19 e 25.

LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In: *Advances in Neural Information Processing Systems*. [s.n.], 2020. v. 33, p. 9459–9474. Disponível em: <https://arxiv.org/abs/2005.11401>. Citado 2 vezes nas páginas 12 e 19.

LIU, N. F. et al. Lost in the middle: How language models use long contexts. *Transactions*

of the Association for Computational Linguistics, v. 12, p. 157–173, 2024. Disponível em: <https://aclanthology.org/2024.tacl-1.9>. Citado na página 18.

MAGESH, V. et al. Hallucination-free? assessing the reliability of leading AI legal research tools. *arXiv preprint arXiv:2405.20362*, 2024. Disponível em: <https://arxiv.org/abs/2405.20362>. Citado 3 vezes nas páginas 13, 18 e 24.

MICROSOFT. *GraphRAG: a modular graph-based Retrieval-Augmented Generation (RAG) system*. 2026. <https://github.com/microsoft/graphrag>. Versão 3.1.0. Acesso em: 13 out. 2026. Citado 2 vezes nas páginas 24 e 27.

MICROSOFT. *GraphRAG documentation*. 2026. <https://microsoft.github.io/graphrag/>. Acesso em: 13 out. 2026. Citado 2 vezes nas páginas 24 e 27.

NEWMAN, M. E. J. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, v. 103, n. 23, p. 8577–8582, 2006. Disponível em: <https://doi.org/10.1073/pnas.0601602103>. Citado na página 21.

OLLAMA. *Ollama*. 2026. <https://ollama.com/>. Acesso em: 13 out. 2026. Citado 3 vezes nas páginas 15, 25 e 43.

OPENAI. *New embedding models and API updates*. 2024. <https://openai.com/index/new-embedding-models-and-api-updates/>. Acesso em: 13 out. 2026. Citado na página 19.

OPENAI. *OpenAI API documentation: models*. 2026. <https://platform.openai.com/docs/models>. Acesso em: 13 out. 2026. Citado na página 25.

PAN, S. et al. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, v. 36, n. 7, p. 3580–3599, 2024. Disponível em: <https://doi.org/10.1109/TKDE.2024.3352100>. Citado na página 21.

PENG, B. et al. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024. Disponível em: <https://arxiv.org/abs/2408.08921>. Citado na página 22.

PLATFORMS, M. *React*. 2026. <https://react.dev/>. Acesso em: 13 out. 2026. Citado na página 25.

RAMÍREZ, S. *FastAPI*. 2026. <https://fastapi.tiangolo.com/>. Acesso em: 13 out. 2026. Citado na página 25.

REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. [s.n.], 2019. p. 3982–3992. Disponível em: <https://aclanthology.org/D19-1410>. Citado na página 19.

RESEARCH, M. *Introducing DRIFT Search: Combining global and local search methods to improve quality and efficiency*. 2024. <https://www.microsoft.com/en-us/research/blog/introducing-drift-search-combining-global-and-local-search-methods-to-improve-quality-and-efficiency/>. Acesso em: 13 out. 2026. Citado 2 vezes nas páginas 22 e 33.

SENNRICH, R.; HADDOW, B.; BIRCH, A. Neural machine translation of rare words with subword units. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. [s.n.], 2016. p. 1715–1725. Disponível em: <https://aclanthology.org/P16-1162>. Citado na página 17.

SQLite Consortium. *SQLite*. 2026. <https://www.sqlite.org/>. Acesso em: 13 out. 2026. Citado na página 25.

Tailwind Labs. *Tailwind CSS*. 2026. <https://tailwindcss.com/>. Acesso em: 13 out. 2026. Citado na página 25.

TRAAG, V. A.; WALTMAN, L.; ECK, N. J. van. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, Nature Publishing Group UK London, v. 9, n. 1, p. 5233, 2019. Disponível em: <https://doi.org/10.1038/s41598-019-41695-z>. Citado na página 21.

VASWANI, A. et al. Attention is all you need. *Advances in neural information processing systems*, v. 30, 2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Citado 2 vezes nas páginas 12 e 17.

Vite Team. *Vite: Next Generation Frontend Tooling*. 2026. <https://vite.dev/>. Acesso em: 13 out. 2026. Citado na página 25.

WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, v. 35, p. 24824–24837, 2022. Disponível em: <https://arxiv.org/abs/2201.11903>. Citado na página 18.

WHATWG. *HTML Living Standard: Server-sent events*. 2026. <https://html.spec.whatwg.org/multipage/server-sent-events.html>. Acesso em: 13 out. 2026. Citado na página 25.

Apêndices

APÊNDICE A – Configuração do GraphRAG Utilizada

Esta listagem reproduz o arquivo `data/settings.yaml` do repositório, utilizado na indexação descrita no Capítulo 4 e nas consultas da aplicação. A chave de API é referenciada por variável de ambiente e não faz parte do arquivo.

Listagem A.1 – Arquivo `settings.yaml`.

```
### GraphRAG v3 - configuração com modelos em nuvem (OpenAI por padrão)
### Alternativa 100% local (Ollama): config/settings.ollama.yaml
### Outros provedores (Anthropic, Google): blocos prontos em config/model_providers.yaml
### ATENÇÃO: o GraphRAG substitui variáveis de ambiente (cifrão + chaves) em TODO o
### arquivo, inclusive nos comentários. Nunca escreva o caractere cifrão em comentários.
### Docs: https://microsoft.github.io/graphrag/config/yaml/
###
### As chaves de API ficam em data/.env (NÃO versionar; modelo em config/.env.example).
### O GraphRAG carrega o .env da pasta --root (/data) automaticamente.
###
### Estratégia de custo-benefício: cada etapa usa o modelo mais barato que a
### atende bem. Os IDs abaixo são referenciados nas seções de workflow.
###   - strong_model  : extração do grafo, relatórios de comunidade, respostas finais
###   - cheap_model   : resumo de descrições (tarefa simples)
###   - embedding     : vetores (Anthropic não tem API de embeddings)

### Temperatura: os modelos GPT-6 (de raciocínio) só aceitam temperature=1.
### Ela é fixada explicitamente em call_args para valer igualmente em todas as
### etapas e modalidades de busca (local, global, drift e basic), evitando que a
### temperatura seja uma variável de confusão na comparação com o baseline.

### LLM settings ###

completion_models:
  strong_model:
    model_provider: openai
    model: gpt-6.1-sol
    auth_method: api_key
    api_key: ${OPENAI_API_KEY}
    concurrent_requests: 8
    async_mode: asyncio
    max_retries: 6
    retry:
      type: exponential_backoff
    call_args:
      temperature: 1
```

```
37        timeout: 180
38
39    cheap_model:
40      model_provider: openai
41      model: gpt-6-luna
42      auth_method: api_key
43      api_key: ${OPENAI_API_KEY}
44      concurrent_requests: 8
45      async_mode: asyncio
46      max_retries: 6
47      retry:
48        type: exponential_backoff
49      call_args:
50        temperature: 1
51        timeout: 120
52
53 embedding_models:
54    default_embedding_model:
55      model_provider: openai
56      model: text-embedding-3-large    # 3072 dimensões; custo desprezível para o
          CDC
57      auth_method: api_key
58      api_key: ${OPENAI_API_KEY}
59      concurrent_requests: 8
60      max_retries: 6
61      retry:
62        type: exponential_backoff
63
64
65 ### Document processing settings ###
66
67 ## Entrada: CDC estruturado (um objeto JSON por artigo), gerado por
68 ## corpus/parser/legislation_parser.py a partir de corpus/raw/cdc_planalto.txt.
69 ## O parser garante ids únicos (ex.: cdc_art_54-A), usados como ids dos
      documentos.
70 input:
71    type: json         # lê todos os arquivos .json de data/input
72    id_column: id
73    title_column: id
74    text_column: text
75
76 ## Cada artigo cabe inteiro em um chunk (maior artigo ~ 900 tokens).
77 ## prepend_metadata injeta a hierarquia da lei no início de cada chunk, dando ao
78 ## LLM o contexto de Título/Capítulo/Seção na extração, e a situação do artigo
79 ## (vigente/vetado/revogado), para que vetos sejam registrados corretamente no
      grafo.
80 chunking:
81    type: tokens
82    size: 1200
83    overlap: 100
84    encoding_model: o200k_base
85    prepend_metadata: [lei_sigla, titulo_nome, capitulo_nome, secao_nome, situacao]
86
87 ### Storage settings ###
88
89 input_storage:
90    type: file
```

```
  base_dir: "input"

output_storage:
  type: file
  base_dir: "output"

reporting:
  type: file
  base_dir: "logs"

cache:
  type: json
  storage:
    type: file
    base_dir: "cache"

vector_store:
  type: lancedb
  db_uri: output/lancedb   # barra normal: o caminho é resolvido dentro do
      container Linux

### Workflow settings (indexação) ###

embed_text:
  embedding_model_id: default_embedding_model

# Etapa mais importante e mais cara: define a qualidade do grafo.
extract_graph:
  completion_model_id: strong_model
  prompt: "prompts/extract_graph.txt"
  # Taxonomia do domínio. As definições de cada tipo e as regras de nomeação
  # ficam em scripts/prompt_tune.py (ENTITY_TYPE_DEFINITIONS), que as insere no
  # prompt de extração. As duas listas precisam ter os mesmos tipos.
  entity_types:
    - consumidor
    - fornecedor
    - orgao_entidade
    - produto
    - servico
    - contrato
    - direito
    - obrigacao
    - pratica_abusiva
    - instituto_juridico
    - sancao_administrativa
    - infracao_penal
    - instrumento_processual
    - dispositivo_legal
  max_gleanings: 1

# Tarefa simples (fundir descrições de uma mesma entidade): modelo barato basta.
summarize_descriptions:
  completion_model_id: cheap_model
  prompt: "prompts/summarize_descriptions.txt"
  max_length: 500

extract_graph_nlp:
```

```
147   text_analyzer:
148     extractor_type: regex_english
149
150 cluster_graph:
151   max_cluster_size: 10
152
153 extract_claims:
154   enabled: false
155   completion_model_id: cheap_model
156   prompt: "prompts/extract_claims.txt"
157   description: "Any claims or facts that could be relevant to information
        discovery."
158   max_gleanings: 1
159
160 # Relatórios de comunidade são a base da busca global: vale usar o modelo forte.
161 community_reports:
162   completion_model_id: strong_model
163   graph_prompt: "prompts/community_report_graph.txt"
164   text_prompt: "prompts/community_report_text.txt"
165   max_length: 2000
166   max_input_length: 8000
167
168 snapshots:
169   graphml: false
170   embeddings: false
171
172 ### Query settings ###
173
174 local_search:
175   completion_model_id: strong_model
176   embedding_model_id: default_embedding_model
177   prompt: "prompts/local_search_system_prompt.txt"
178
179 # A busca global faz uma chamada "map" por lote de relatórios; o custo é dominado
180 # por essas chamadas. Se quiser economizar, troque para cheap_model.
181 global_search:
182   completion_model_id: strong_model
183   map_prompt: "prompts/global_search_map_system_prompt.txt"
184   reduce_prompt: "prompts/global_search_reduce_system_prompt.txt"
185   knowledge_prompt: "prompts/global_search_knowledge_system_prompt.txt"
186
187 # Os modelos GPT-6 (de raciocínio) só aceitam temperature=1; o padrão da busca
188 # DRIFT é 0, o que gera erro "Unsupported value: 'temperature'".
189 drift_search:
190   completion_model_id: strong_model
191   embedding_model_id: default_embedding_model
192   local_search_temperature: 1
193   reduce_temperature: 1
194   prompt: "prompts/drift_search_system_prompt.txt"
195   reduce_prompt: "prompts/drift_reduce_prompt.txt"
196
197 basic_search:
198   completion_model_id: strong_model
199   embedding_model_id: default_embedding_model
200   prompt: "prompts/basic_search_system_prompt.txt"
```